CMPS453 Cloud Computing
Virtualization

Ayman AboElHassan, PhD
Assistant Professor
ayman.abo.elmaaty@eng.cu.edu.eg

What is Virtual Machine (VM)?

Software

- Run applications ➔ Not tied to a physical machine
- Has its own independent OS (Guest OS)

Why Virtualization?

Foundation of all cloud services ➔ Cloud v1.0

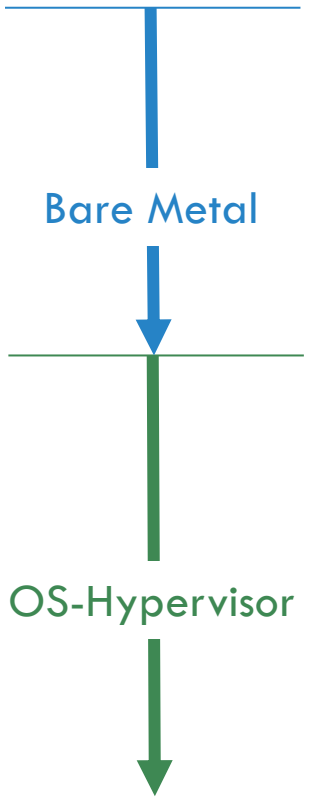
- Hide the HW detail
- Deploy any configuration
- Reduce set-up time
- Multiple Jobs could share same resource

A bit of history

- 1950s Virtual memory based on paging
- 1960s IBM introduced CP/CMS ➔ Time-Sharing OS
- 1970s IBM released VM/370 OS
- 1990s Java Virtual Machine (JVM) introduced

A bit of history

- 1999 VMware launched x86-based virtualization products
- 2003 Xen Open-source
- 2006 Kernel-based Virtual Machine (KVM) Open-sourced
- 2008 MS added Hyper-V to Windows Server 2008 R2
- 2008 Linux Container (LXC) launched
- 2011 IBM, RedHat, HP, and Intel formed Virtualization Alliance to accelerate KVM adoption
- 2013 Docker was launched
- 2018 Podman was launched



Virtualization

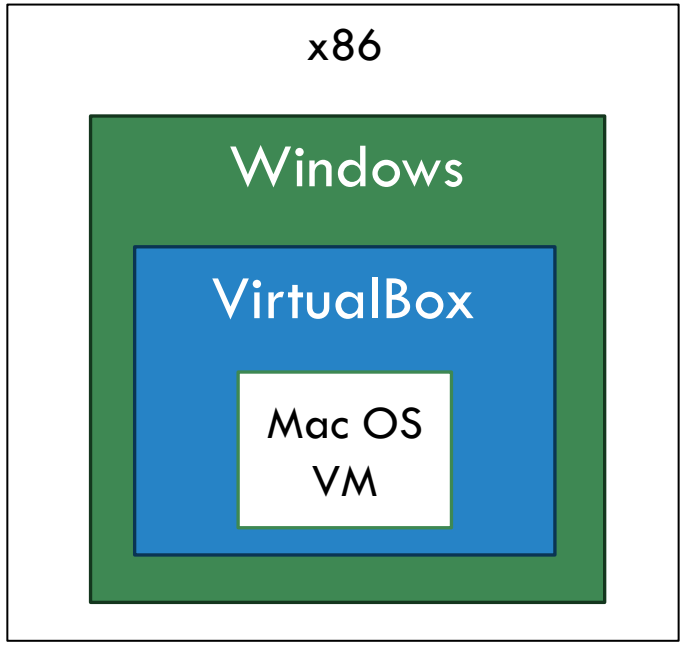
What is Virtualization?

Allows OSs and applications to run on VMs

What is Virtualization?

Allows OSs and applications to run on VMs

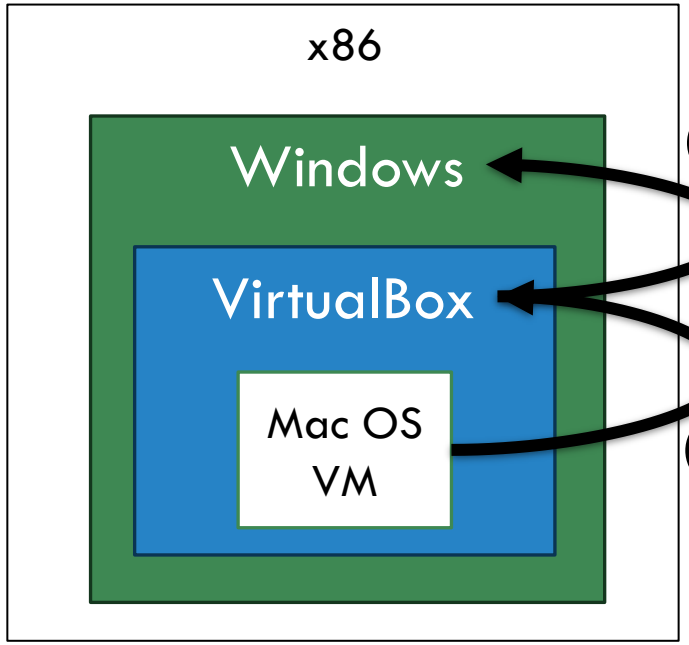
1. Simulate HW functionalities



What is Virtualization?

Allows OSs and applications to run on VMs

1. Simulate HW functionalities



2 imul eax, ebx

Translate the multiplication to the physical machine version

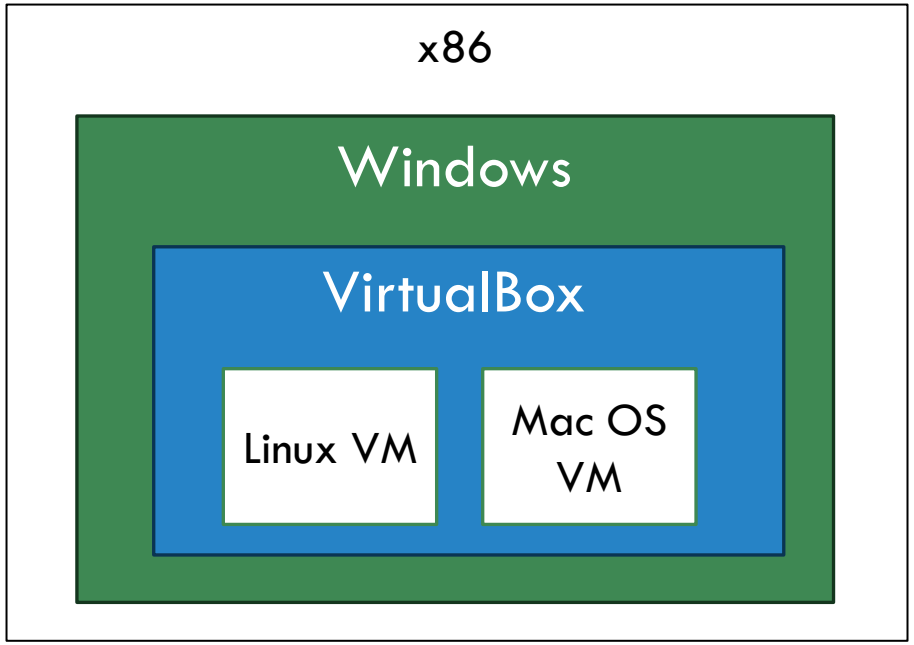
1 mul x0, x0, x1

Multiply values in registers X0 & X1 and save result in X0

What is Virtualization?

Allows OSs and applications to run on VMs

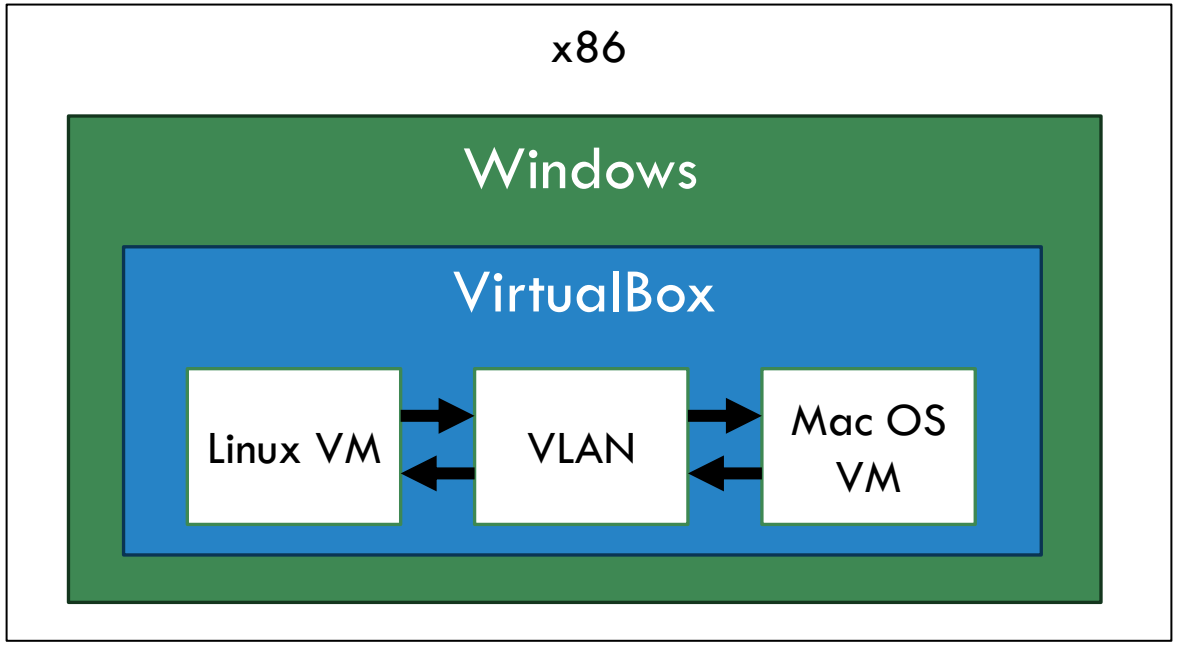
2. Allows Multiple VMs to run independent OSs ➔ Isolation



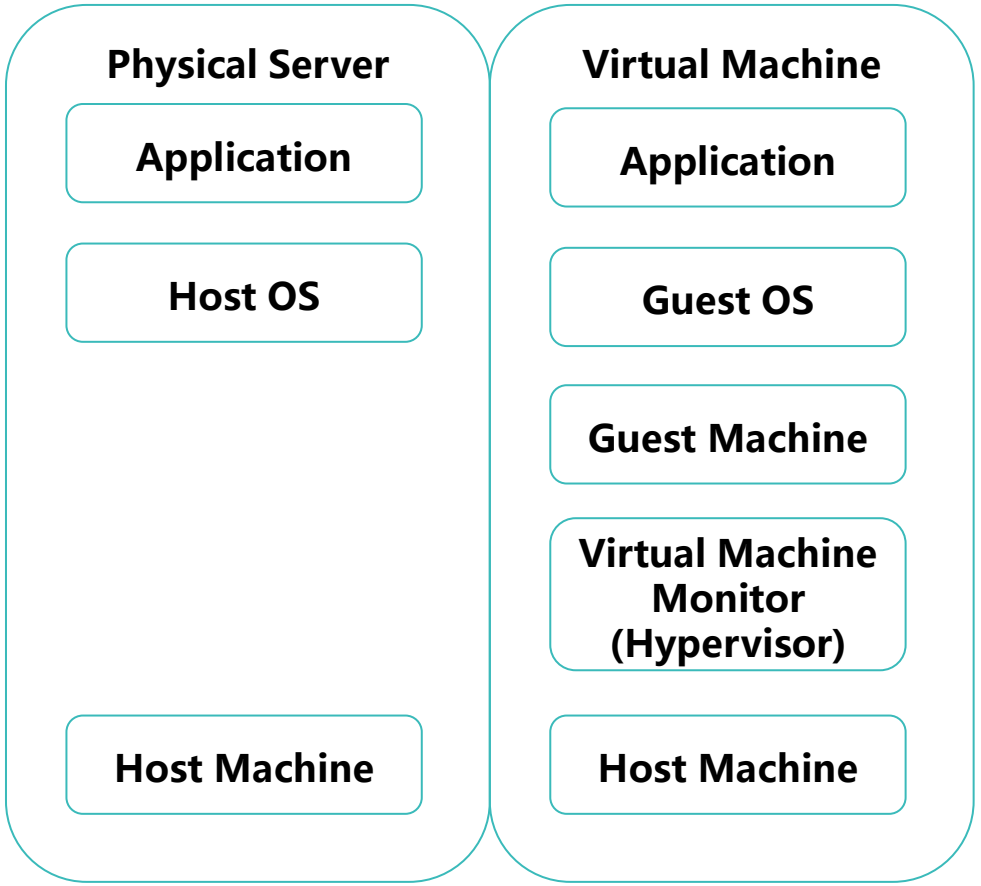
What is Virtualization?

Allows OSs and applications to run on VMs

3. Allows Multiple VMs on a physical machine concurrently



Virtualization Concepts



Host Machine:

Physical machine

Host OS:

Operating system running on the physical machine

Guest Machine:

Virtual machine created through virtualization

Guest OS:

Operating system running in the virtual machine (VM)

Hypervisor:

Virtualization software layer, or

Virtual Machine Monitor (VMM)

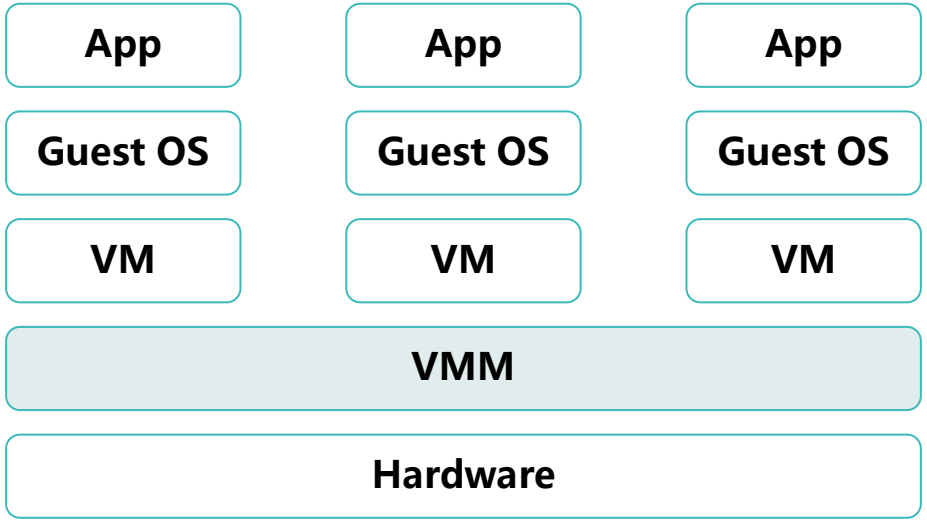
Hypervisor (VMM)

Create and manage virtual environments

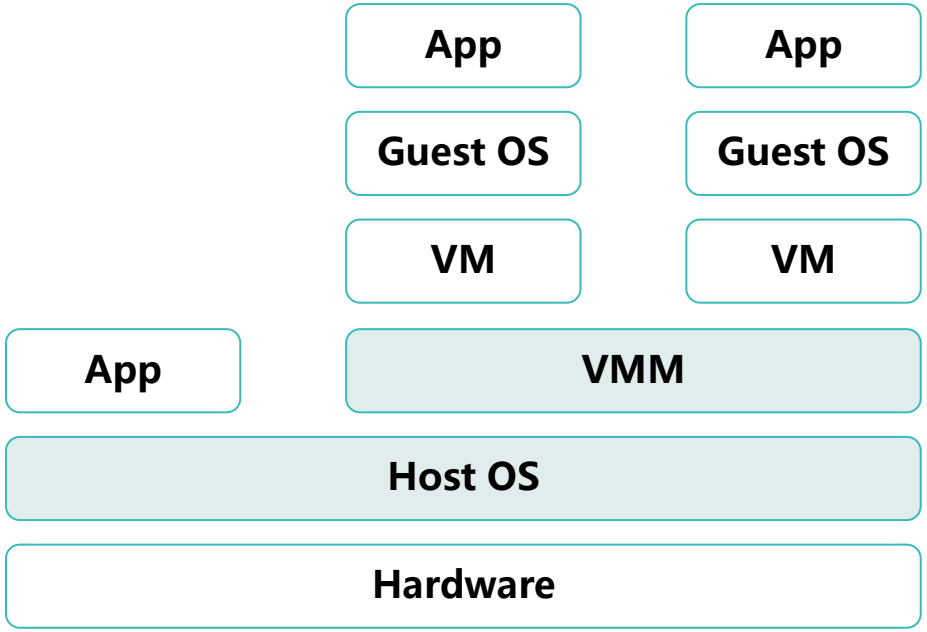
1. Identify, capture, and respond to CPU instructions and HW requests
2. Schedule VM requests queue
3. Return physical machine processing results to corresponding VM

- Complete OS for controlling VMs and resources sharing

Virtualization Types



Bare-Metal Virtualization (Type 1)



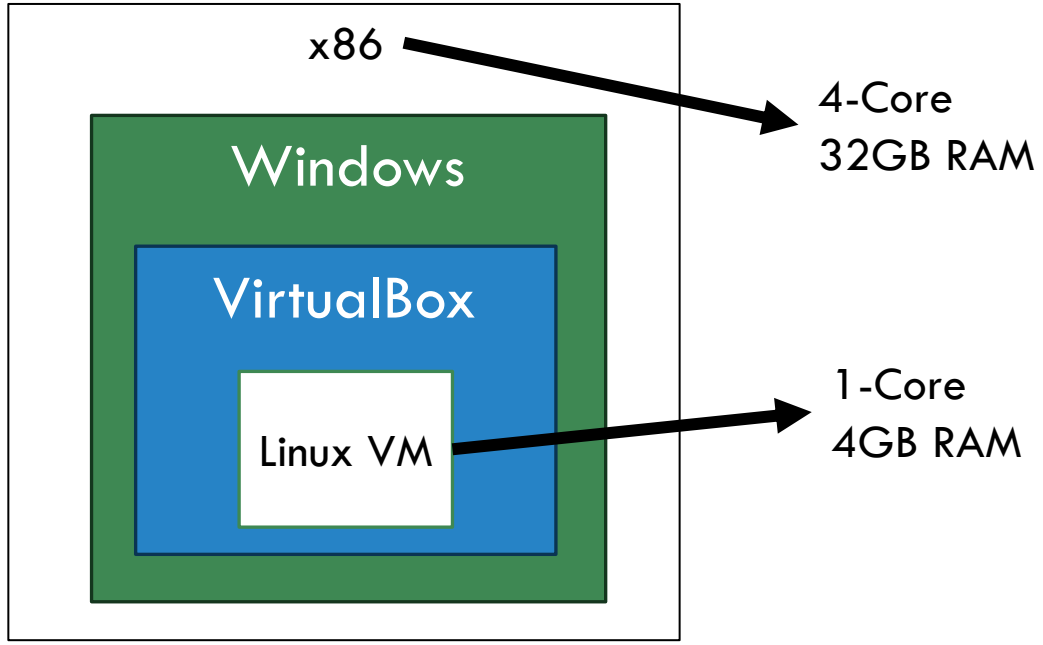
Hosted Virtualization (Type 2)

Virtualization Types

	Bare-Metal Virtualization	Hosted Virtualization
	Customized Host OS	Common application
Runs on	HW directly	Host OS
Best for	Large resource-intensive workloads	Desktop & dev environments
Knowledge Required	System administrator-level knowledge	Basic user knowledge
Example	Hyper-V, KVM, …	VirtualBox, VMware workstation, …

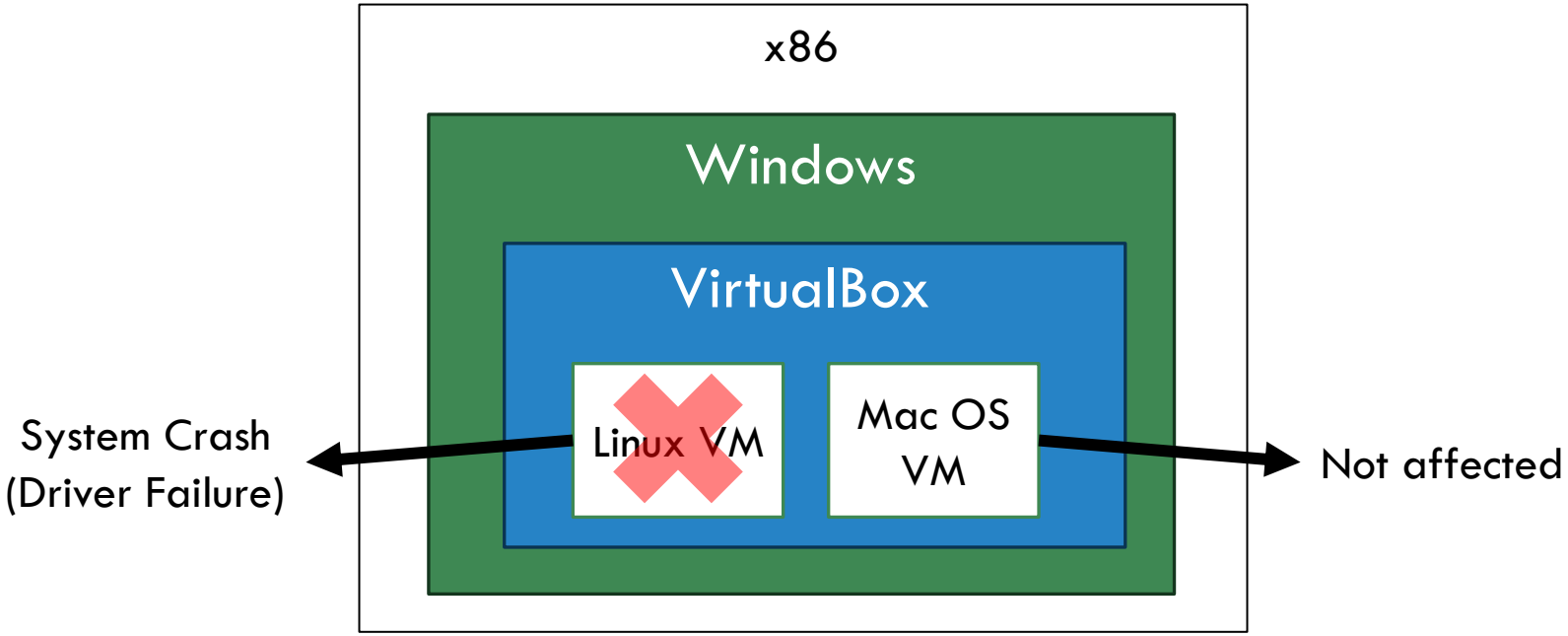
Virtualization Characteristics

1. Partitioning
   - Allocating server resources to multiple VMs ➔ Prevent overuse
   - Each VM access its virtual HW only provided by VMM ➔ HW Quota



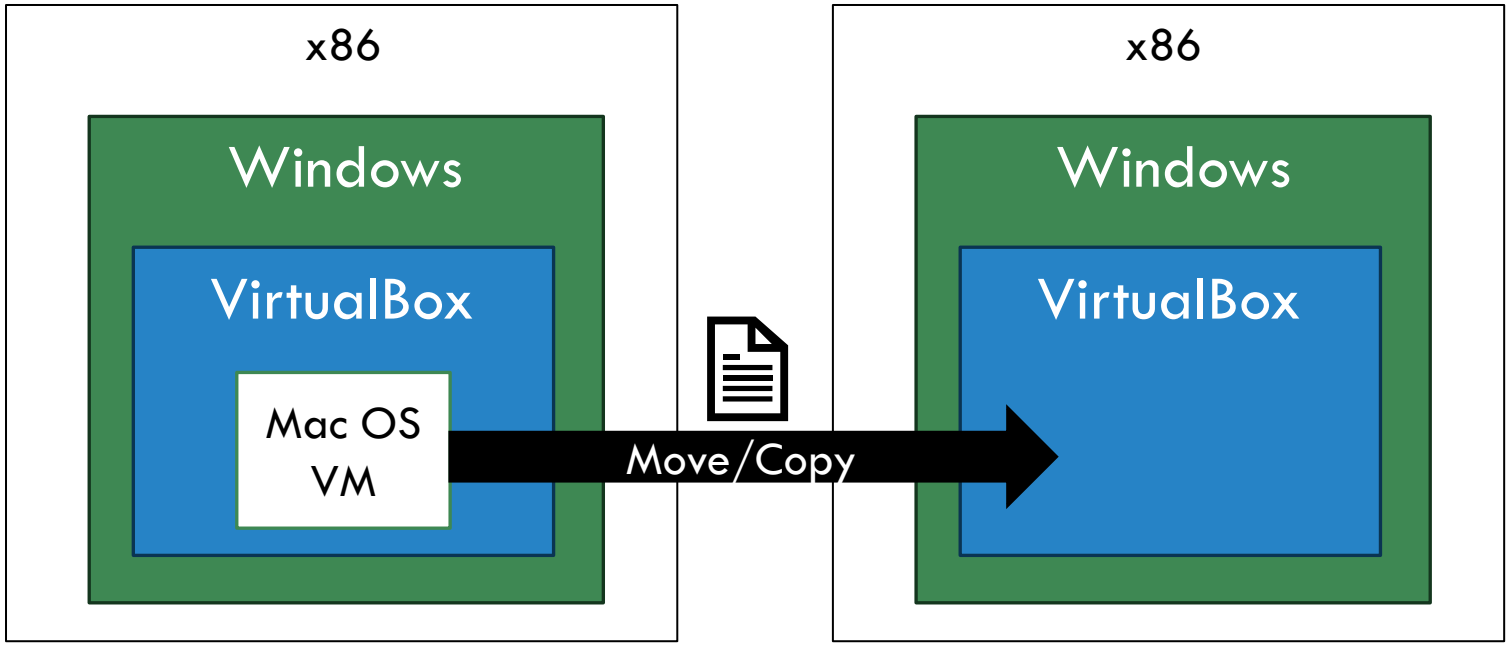
Virtualization Characteristics

2. Isolation
   - VMs and virtual resources are logically isolated



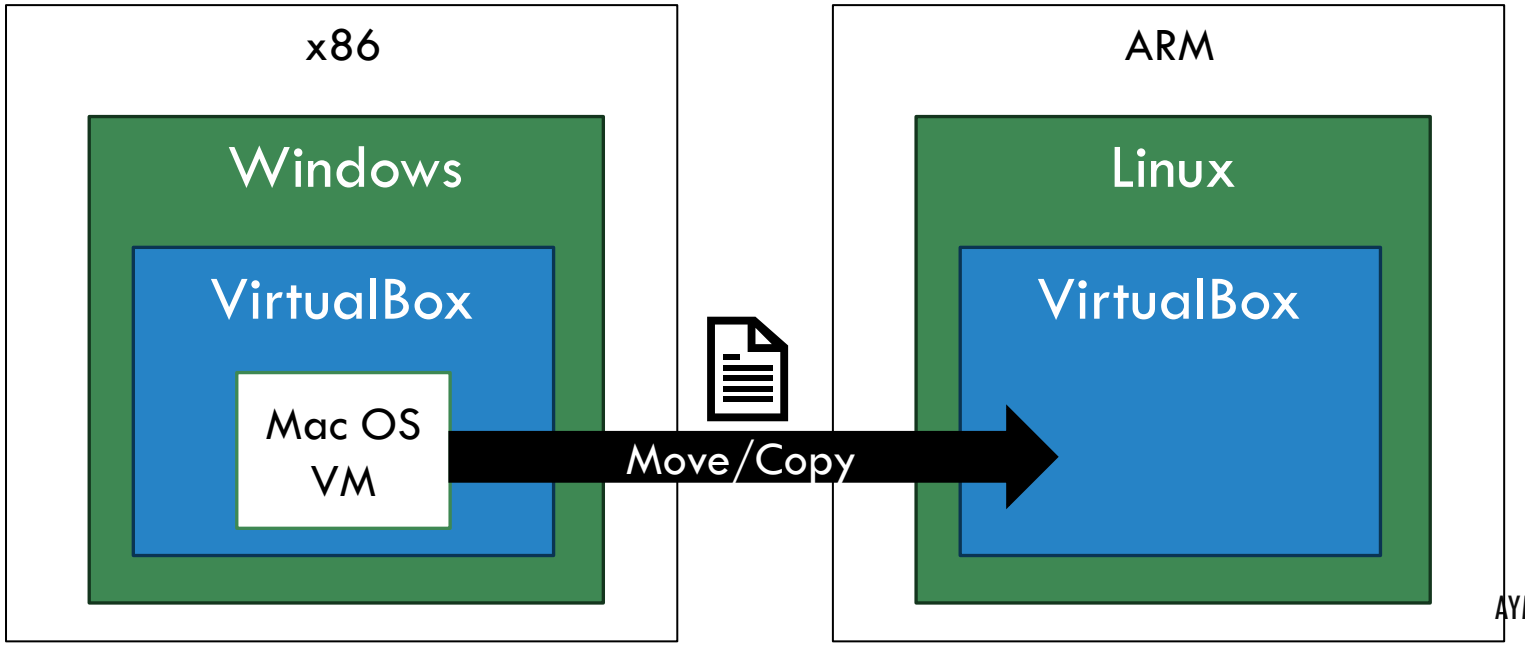
Virtualization Characteristics

3. Encapsulation
   - VM is encapsulated as a group of HW-independent files (.vbox, .vdi)
   - Allows VM migration ➔ Move to other physical machines



Virtualization Characteristics

4. HW Independence
   - VMM manages the VM operations independently from host machine
   - A VM can be migrated from a host machine to another as long as the same VMM running on the target host machine



Conditions for Efficient Virtualization

Popek and Goldberg 1974

"Formal Requirements for Virtualizable Third Generation Architectures"

1. Program running **inside VM** should behave identically when running on an **equivalent machine** directly
2. VMM should be in complete control of virtualized resources
3. Statistically **significant fraction** of machine instructions must be executed without VMM intervention

CPU Virtualization

CPU Virtualization

- How to execute a MacOS command on x86 Windows?
- How to isolate VM instructions from other VMs?
- How to prevent malicious behavior from affecting host?

Instruction Types

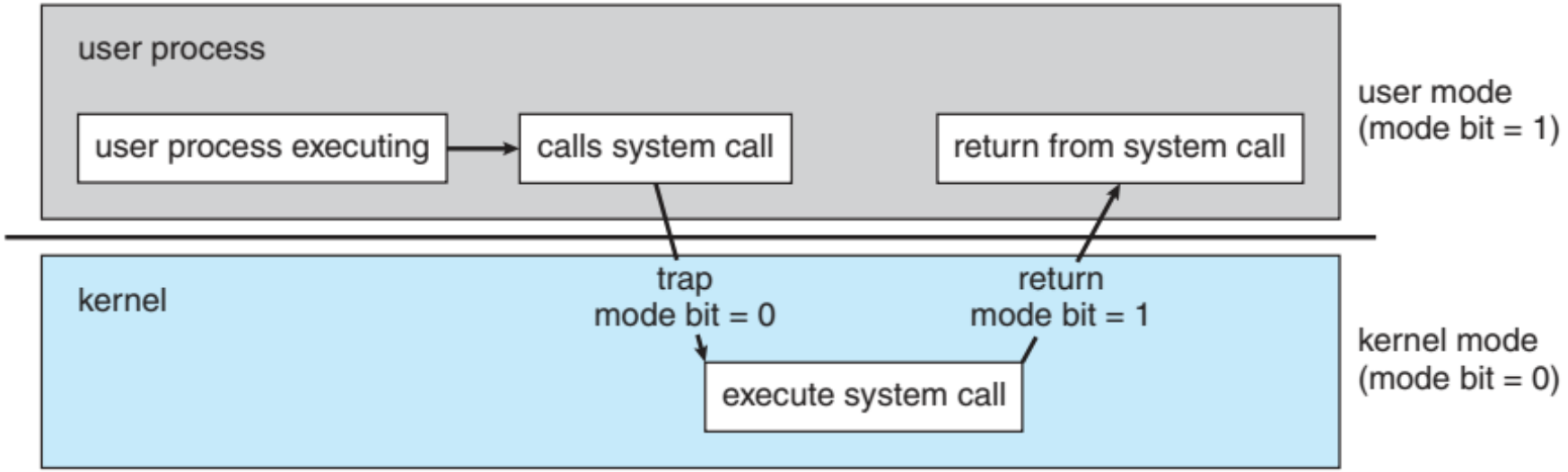
Instructions are either

1. Non-privileged Instructions (i.e.: +, -, *, /, …)
   - Runs in user-mode directly
2. Privileged Instructions (System Calls: getDate(), int 32, …)
   - Must run in kernel-mode
   - If application (user-mode) tries to run that instruction
     - Triggers SW interrupt “Trap” & transfer control to OS

Instruction Types

Instructions are either

1. Non-privileged Instructions (i.e.: +, -, *, /, …)
2. Privileged Instructions (System Calls: getDate(), int 32, …)



Instruction Types

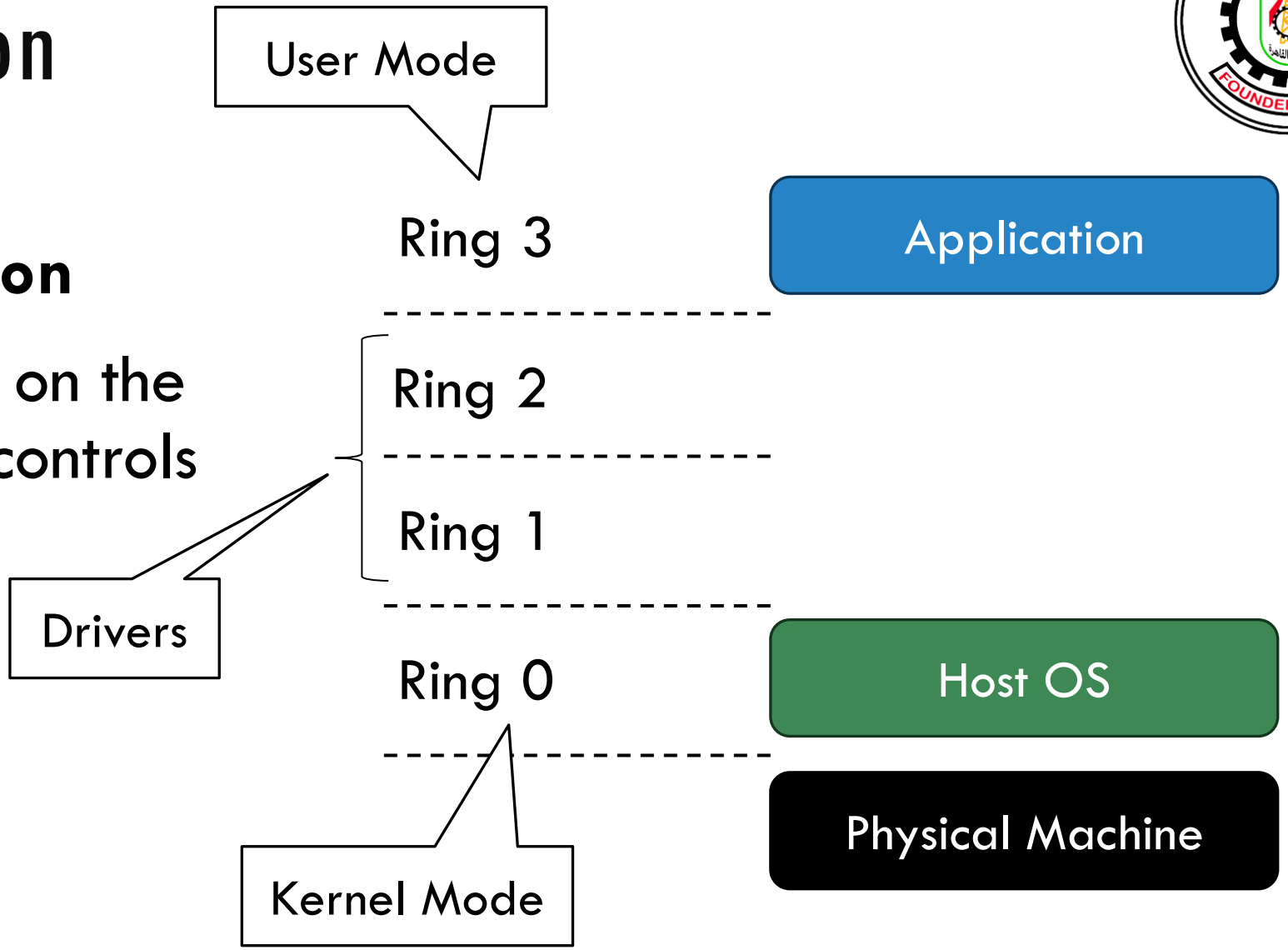
Instructions are either

1. <u>Non-sensitive</u> Instructions (i.e.: +, -, *, /, …)
   - Does not change HW settings
2. <u>Sensitive</u> Instructions (i.e.: int 32, push, pop, …)
   - Instructions that change HW settings ➔ Control Sensitive
   - **OR** behaves differently based HW settings ➔ Behavior Sensitive

CPU Virtualization

Without Virtualization

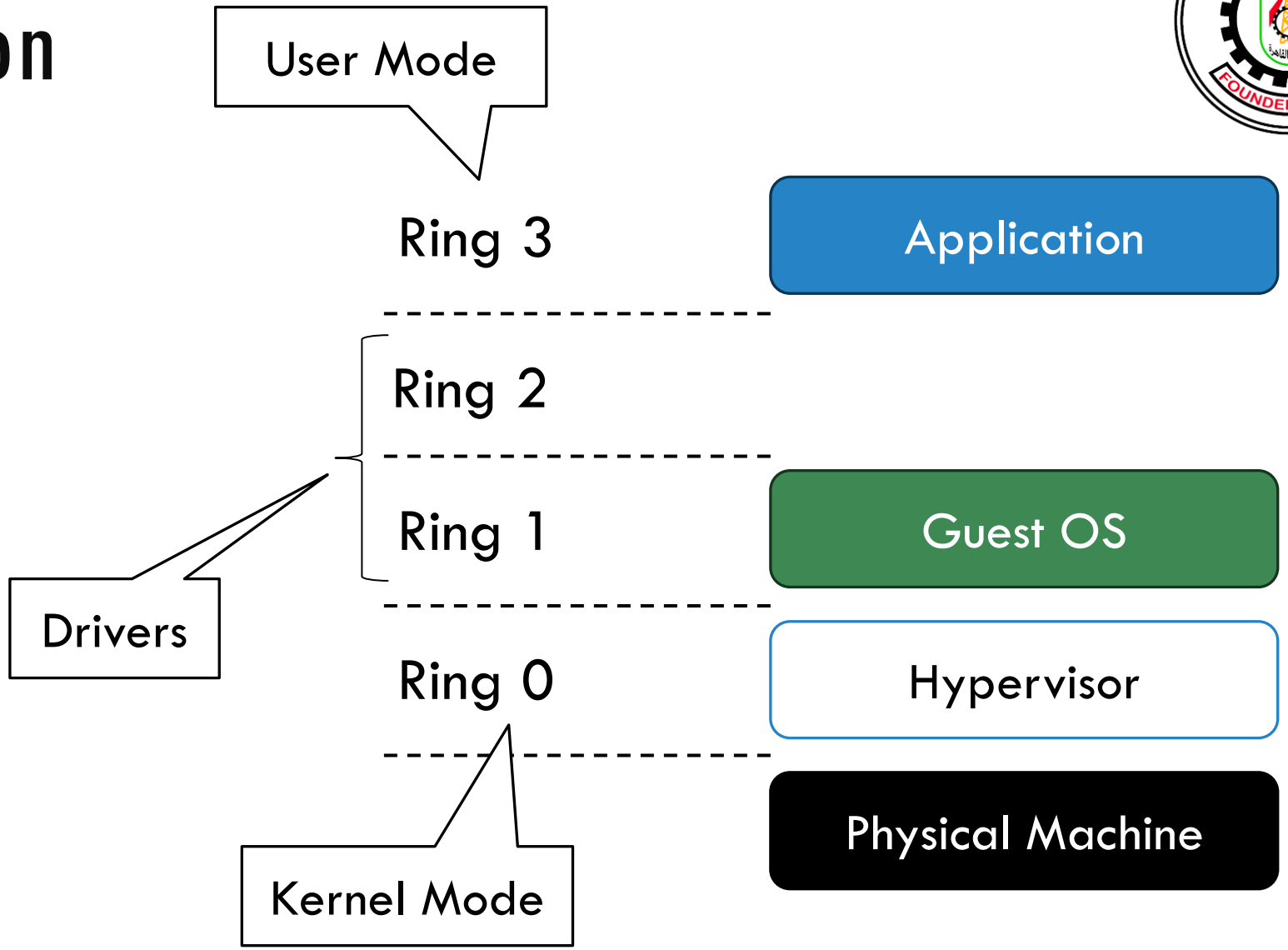
Host OS run directly on the physical machine & controls HW access



CPU Virtualization

With Virtualization

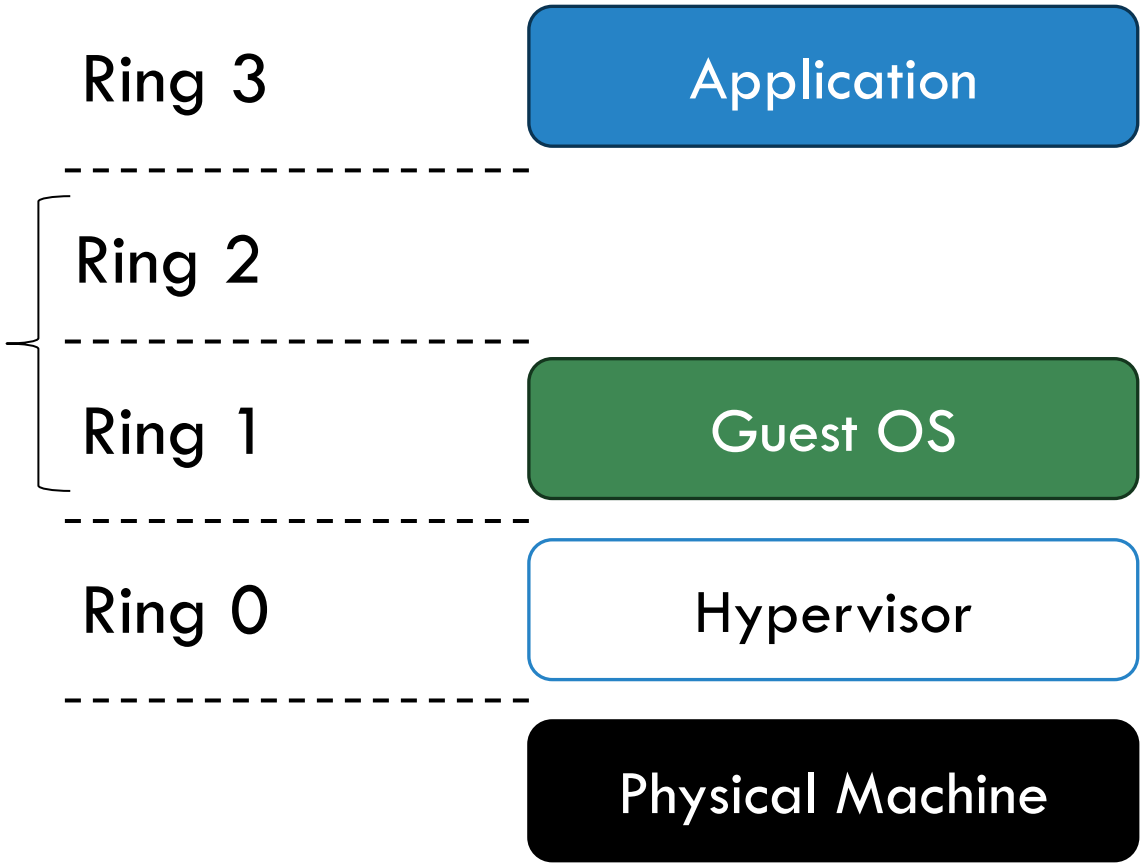
Hypervisor controls access to HW



CPU Virtualization

1. Full Virtualization

Binary Translation

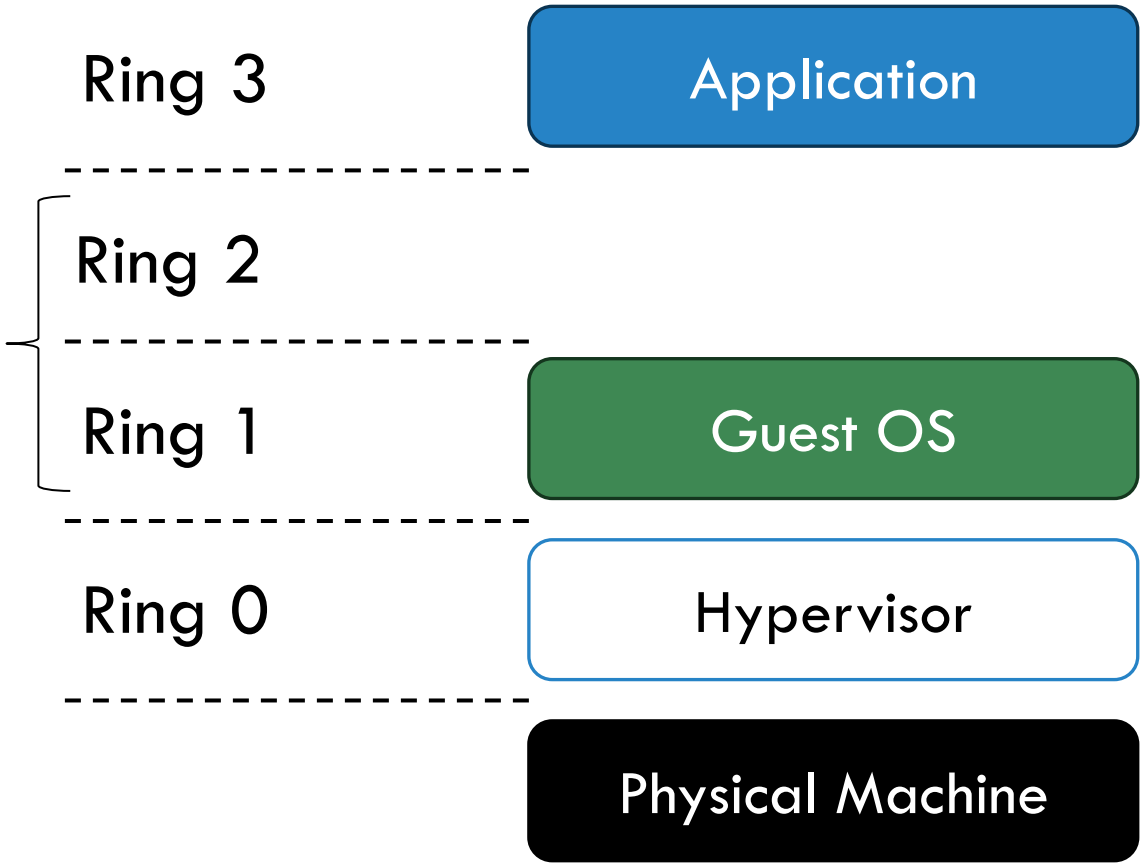


CPU Virtualization

1. **Full Virtualization**

   Binary Translation

   1. Application inside VM runs normal instruction(i.e.: CPU operation as Add, Mul, …)



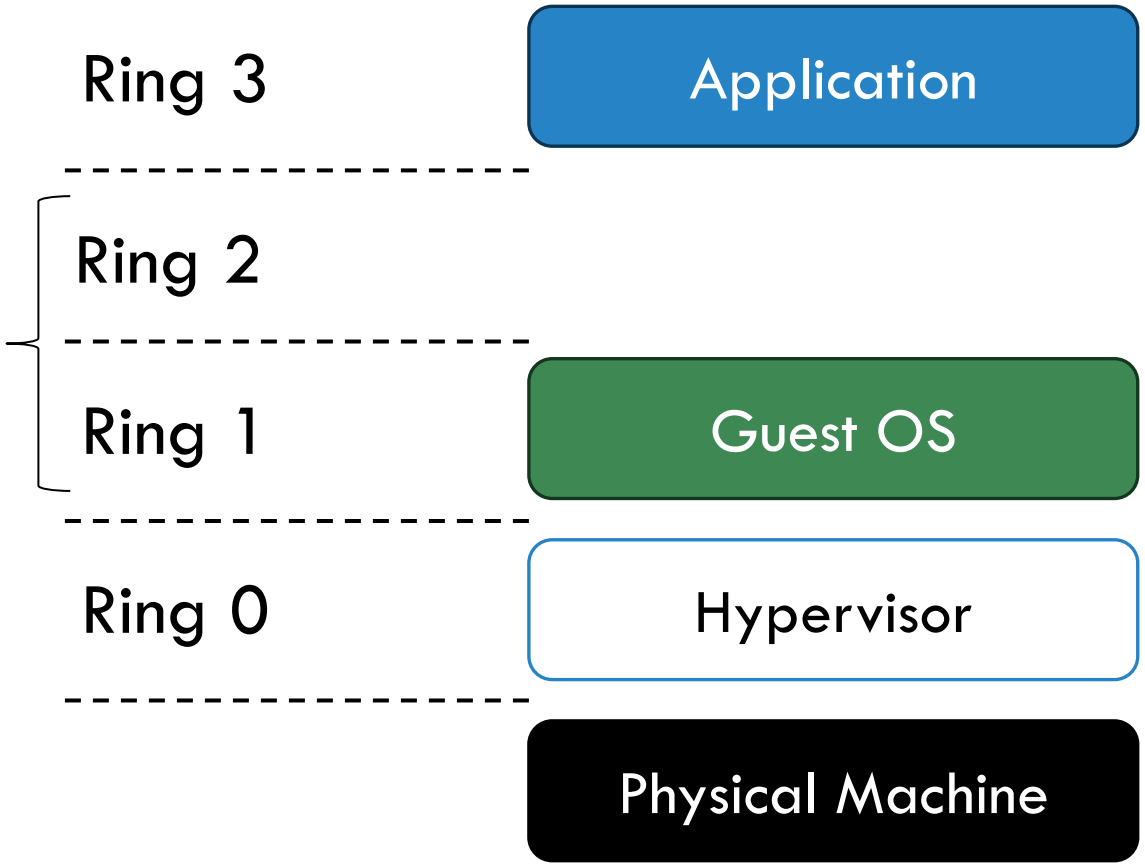
CPU Virtualization

1. **Full Virtualization**

   Binary Translation

   1. Application inside VM runs normal instruction(i.e.: CPU operation as Add, Mul, …)

2 Cases
- Same ISA
- Different ISA



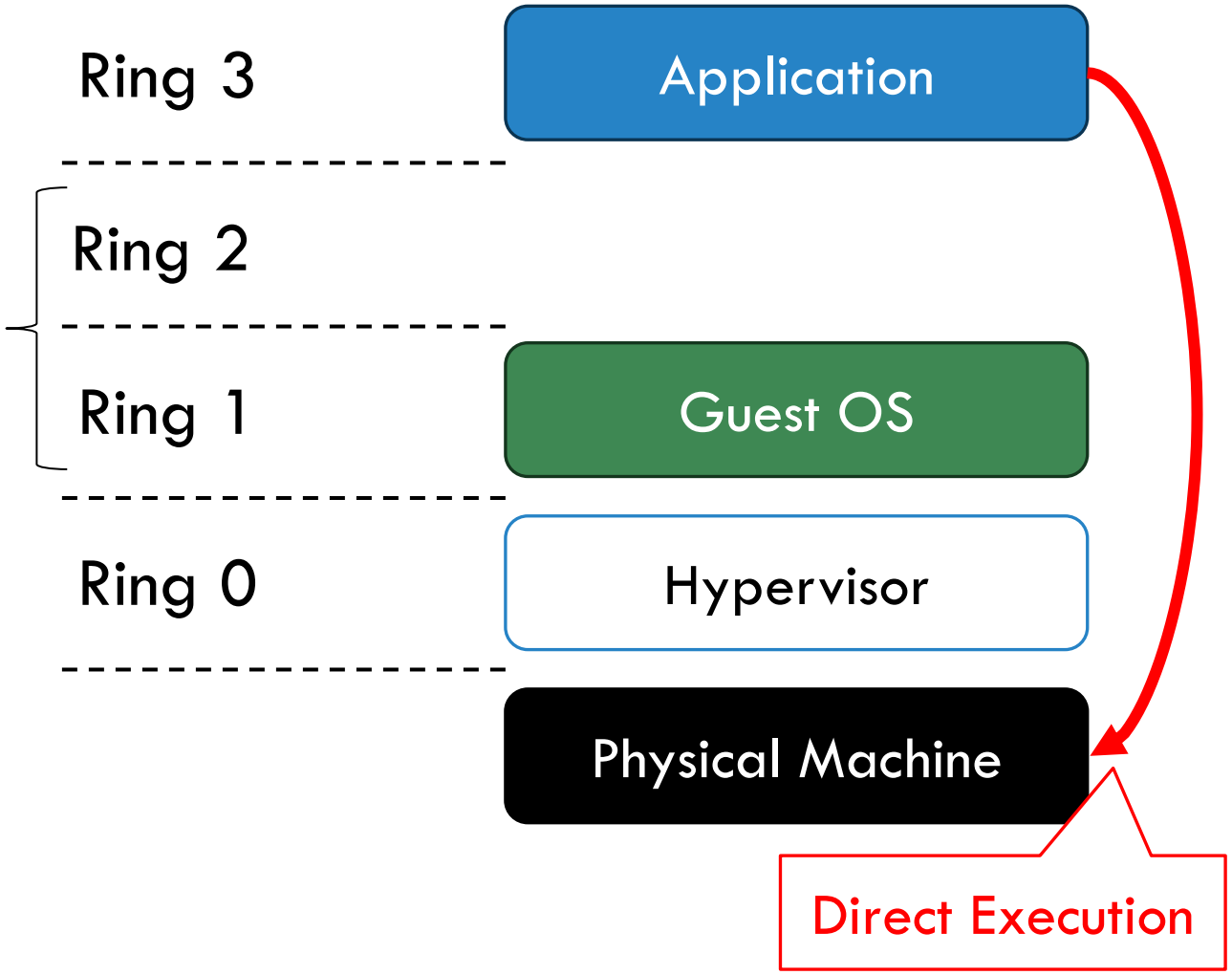
CPU Virtualization

1. **Full Virtualization**

   Binary Translation

   1. Application inside VM runs normal instruction(i.e.: CPU operation as Add, Mul, …)

2 Cases
- Same ISA
- Different ISA



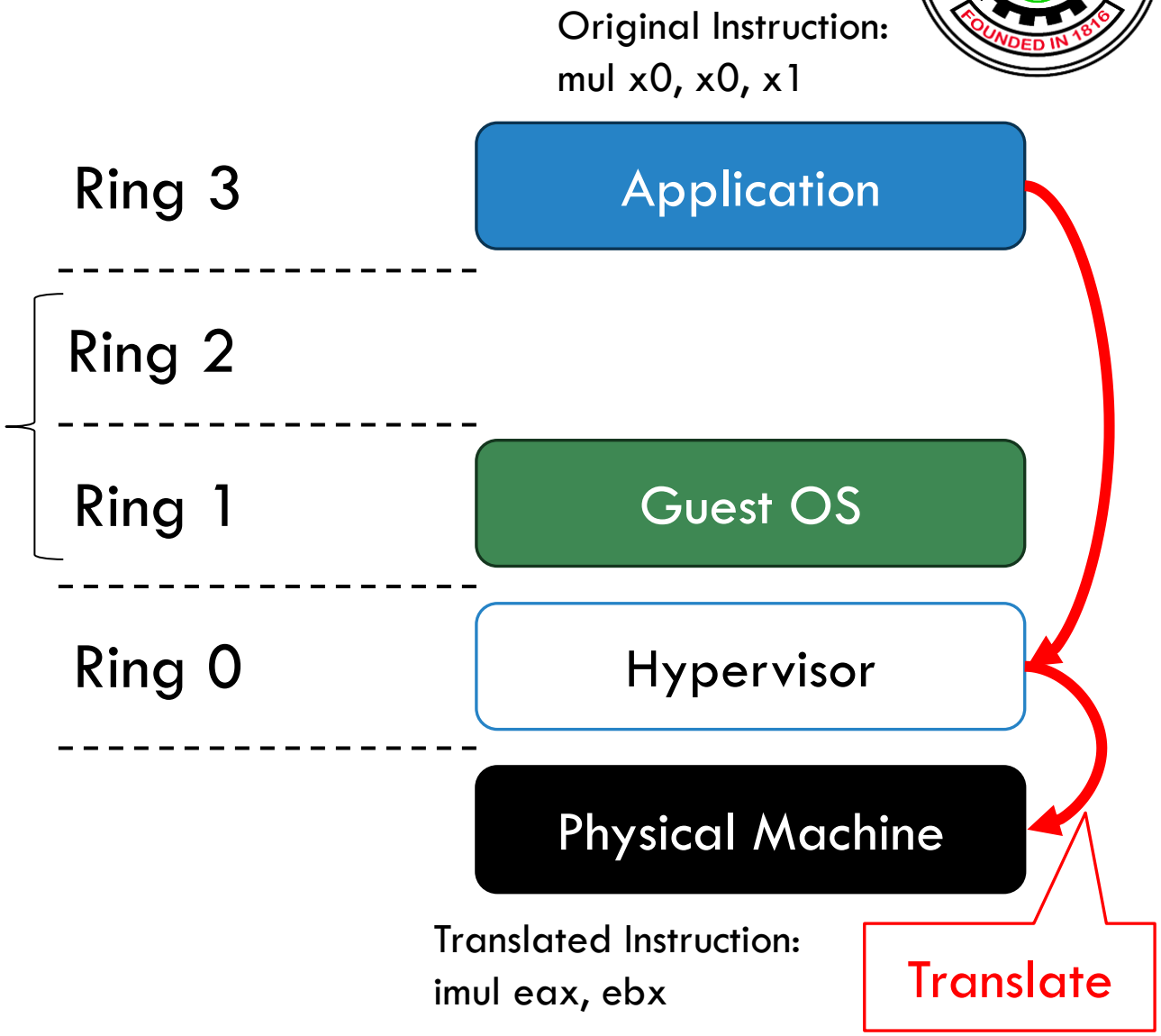
CPU Virtualization

1. **Full Virtualization**

   Binary Translation

   1. Application inside VM runs normal instruction(i.e.: CPU operation as Add, Mul, …)

2 Cases
- Same ISA
- Different ISA



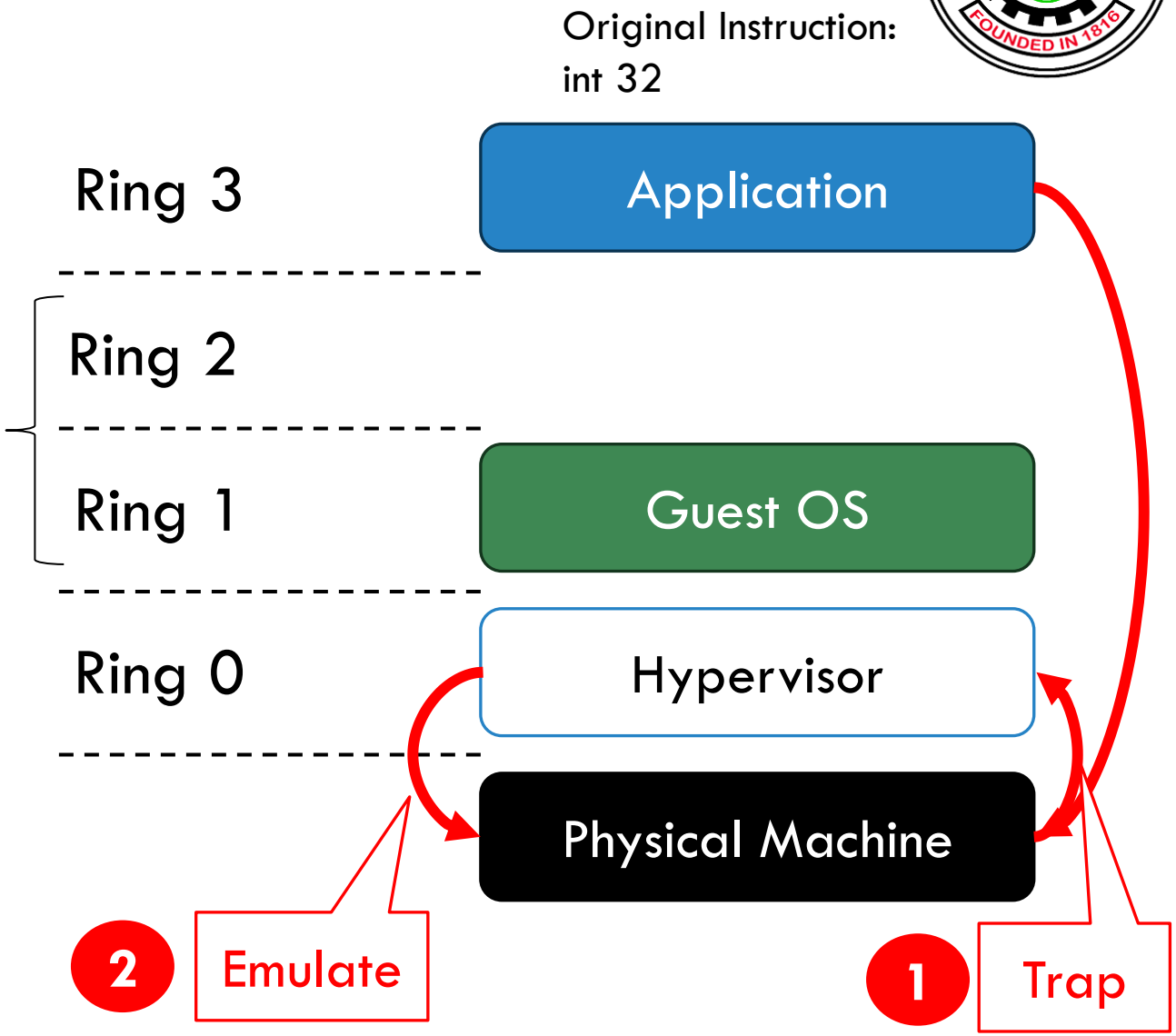
CPU Virtualization

1. **Full Virtualization**

   Binary Translation

   2. Application inside VM runs System Call instruction (i.e.: read file, get mouse XY)

Assuming Same ISA



Simulate Guest OS Behavior

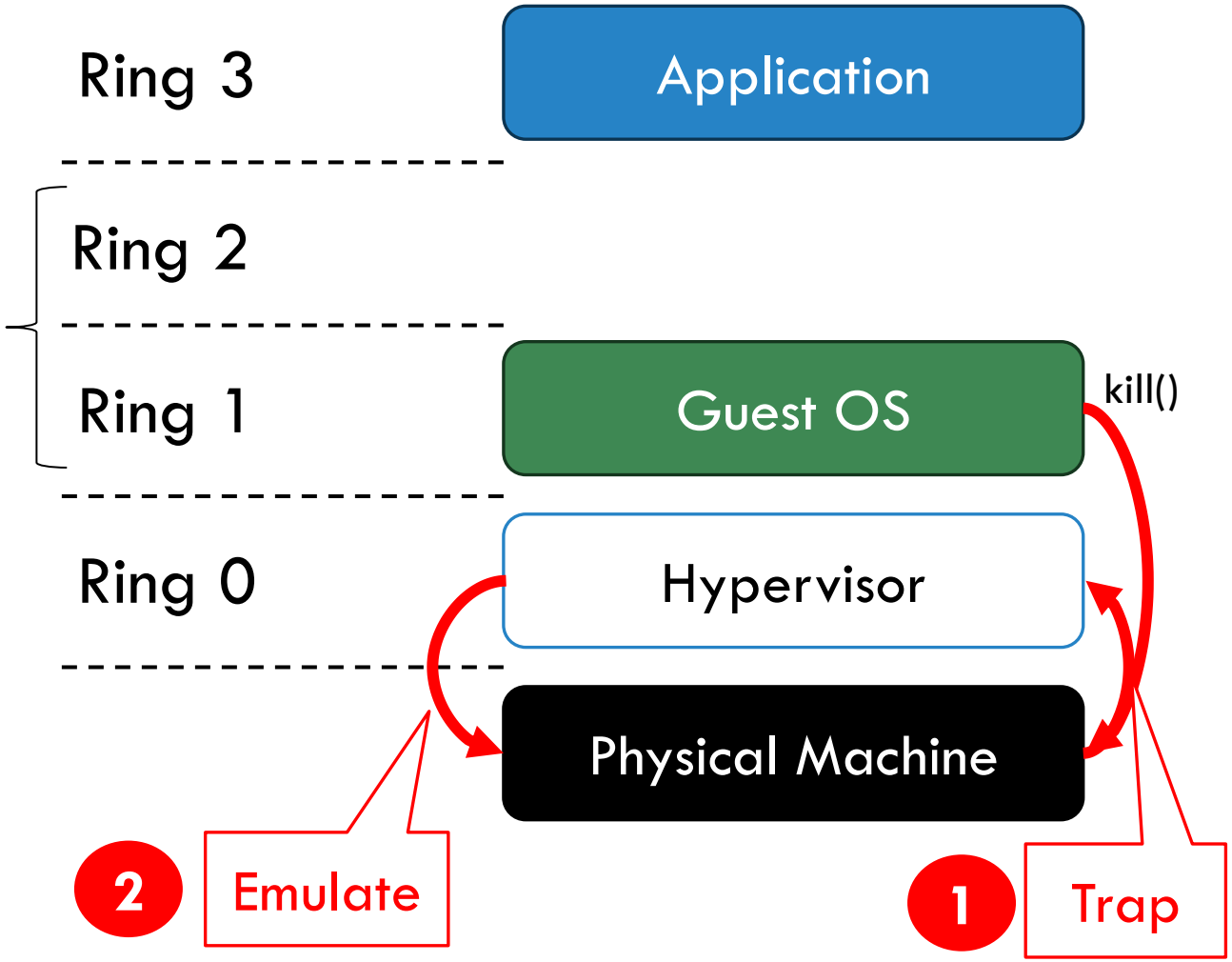
CPU Virtualization

1. **Full Virtualization**

   Binary Translation

   3. Guest OS wants to run a kernel-mode instruction (i.e.: scheduler, start process)

Assuming Same ISA



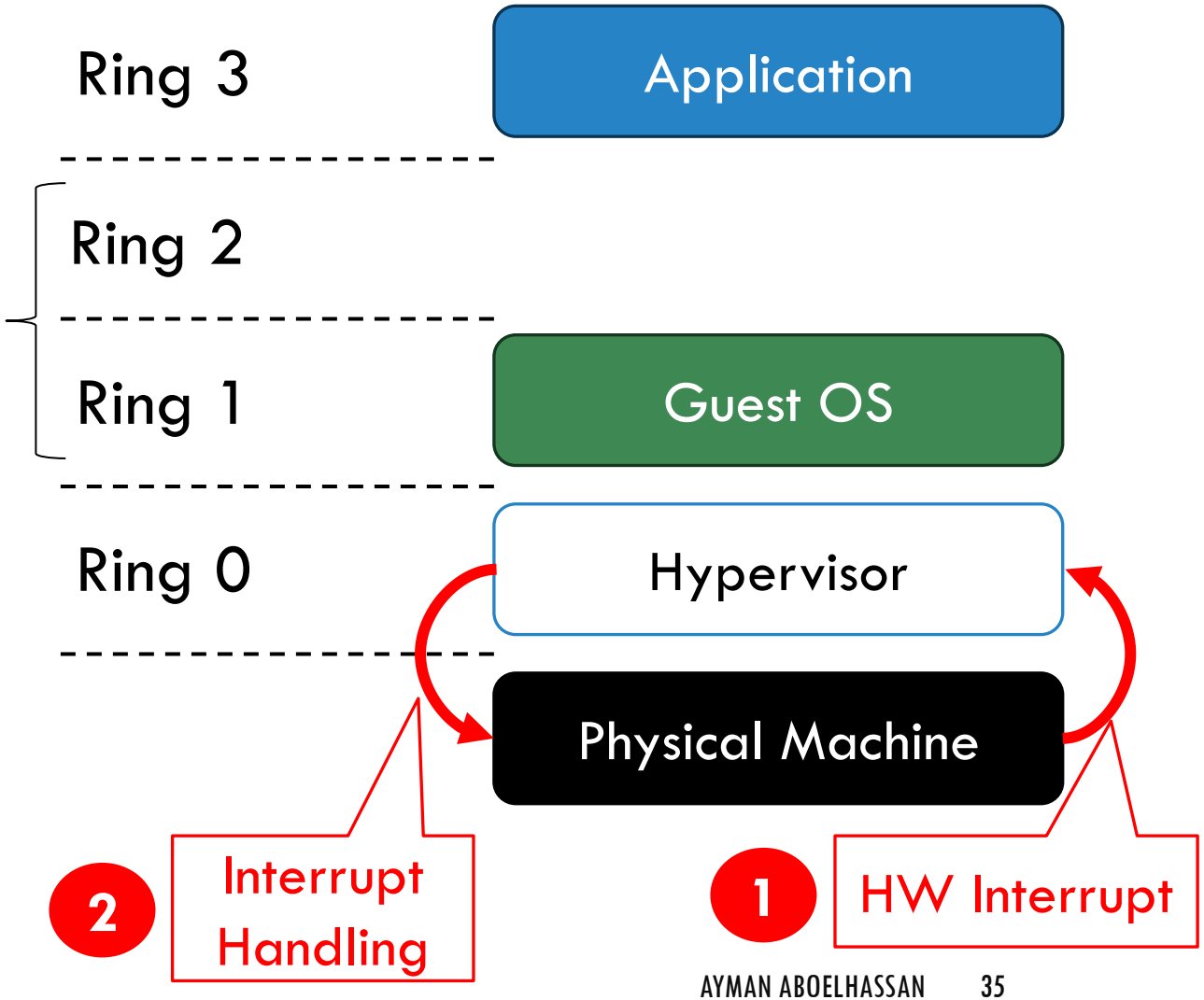
Simulate Guest OS Behavior

CPU Virtualization

1. **Full Virtualization**

   Binary Translation

   4. Hardware interrupts are handled by the VMM (i.e.: Keyboard input)

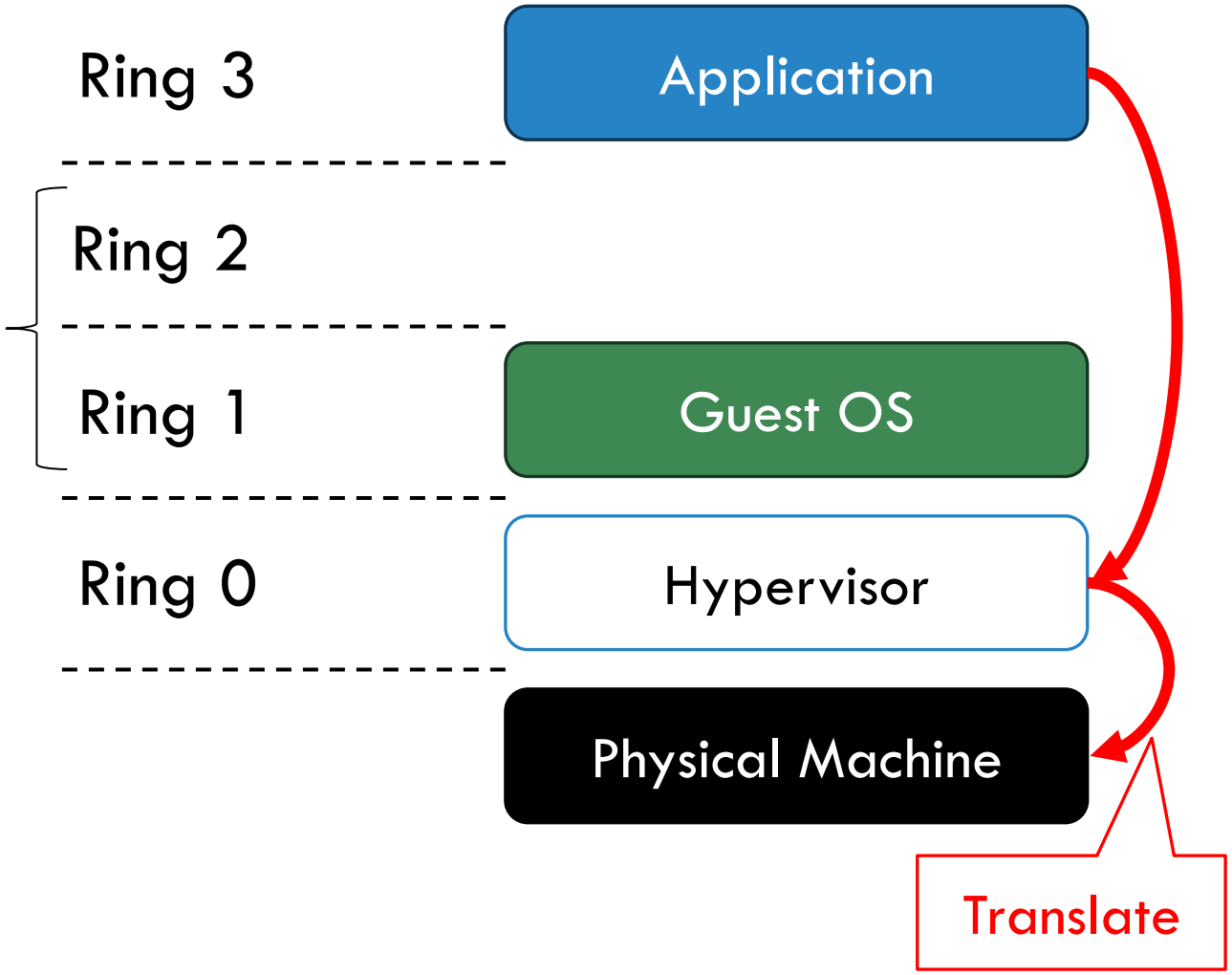


CPU Virtualization

1. **Full Virtualization**

   Binary Translation

   5. Application inside VM runs sensitive non-privileged instruction (i.e.: POP instruction)

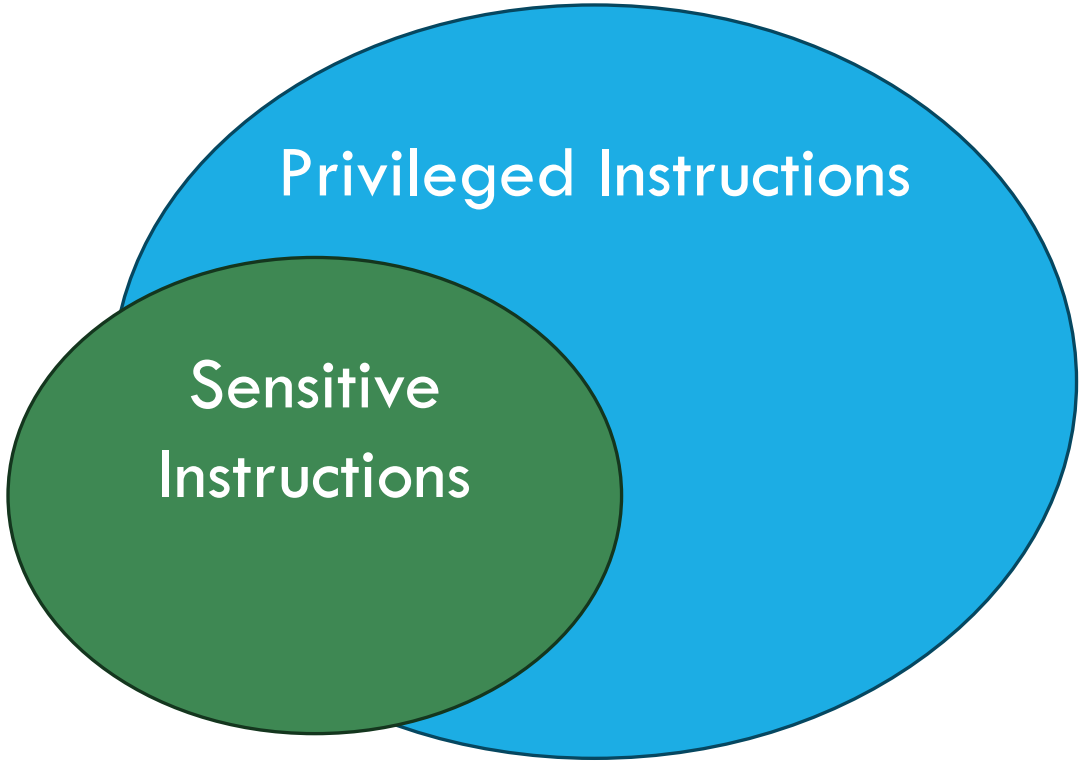


CPU Virtualization

1. Full Virtualization

Binary Translation

- **Potential Problem**
  - Not all sensitive instructions triggers SW interrupt "trap"
    - Pentium processor has 17 functions that are sensitive and non-privileged
  - VMM must detect if any of those functions are called and emulate the VM behavior



CPU Virtualization

1. Full Virtualization

Binary Translation

- Example: Vmware EX server

Pros ➔ No HW or OS modifications
Host isolation & security

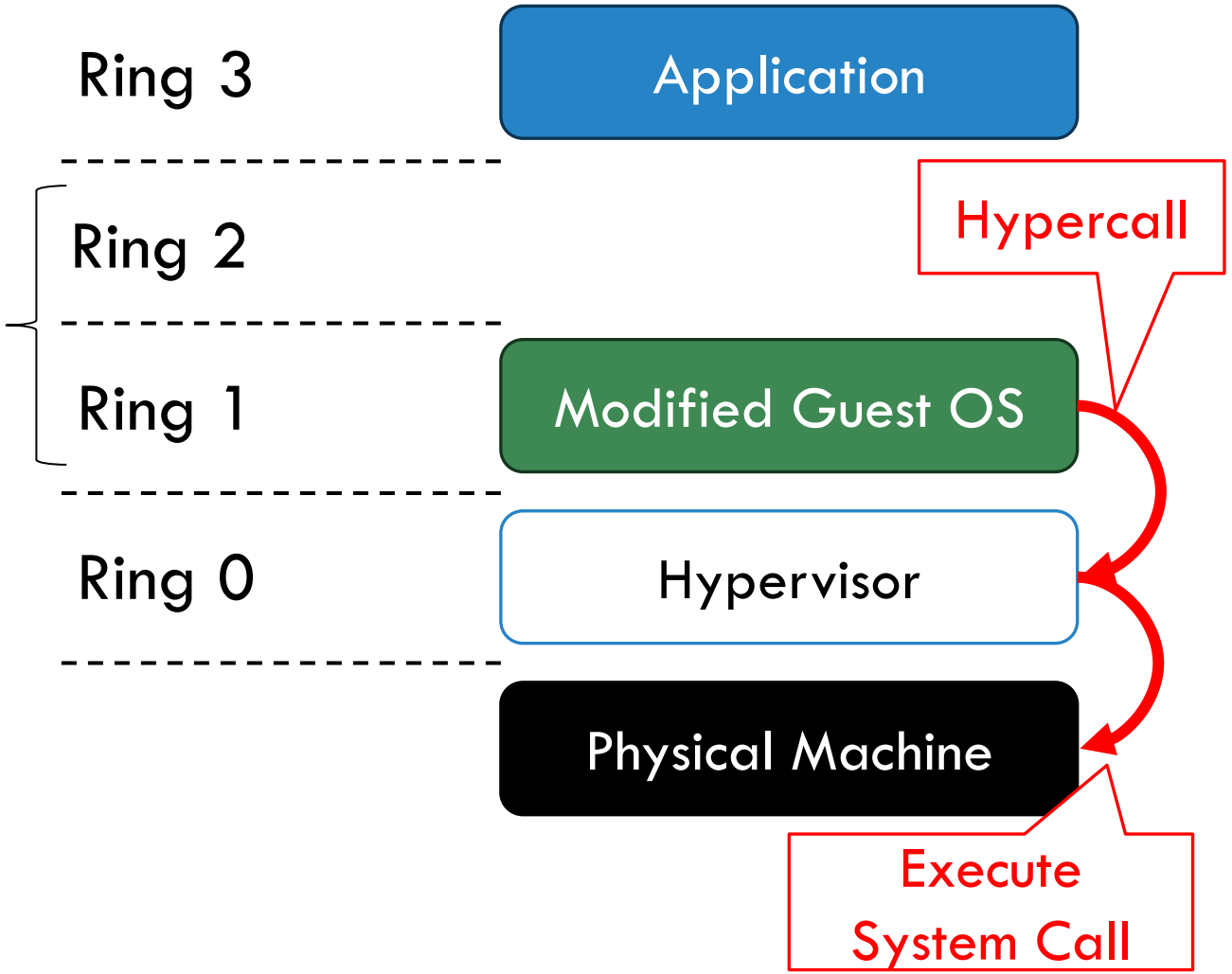
Cons ➔ Slow execution due to translation
VMM Complexity

CPU Virtualization

2. Paravirtualization

Modify **guest OS** to be <u>aware</u> it is <u>virtualized</u>

- Guest OS uses "Hypercall" to replace System Calls
- VMM provide Hypercall API for critical kernel operations as memory management, interrupt handing, …

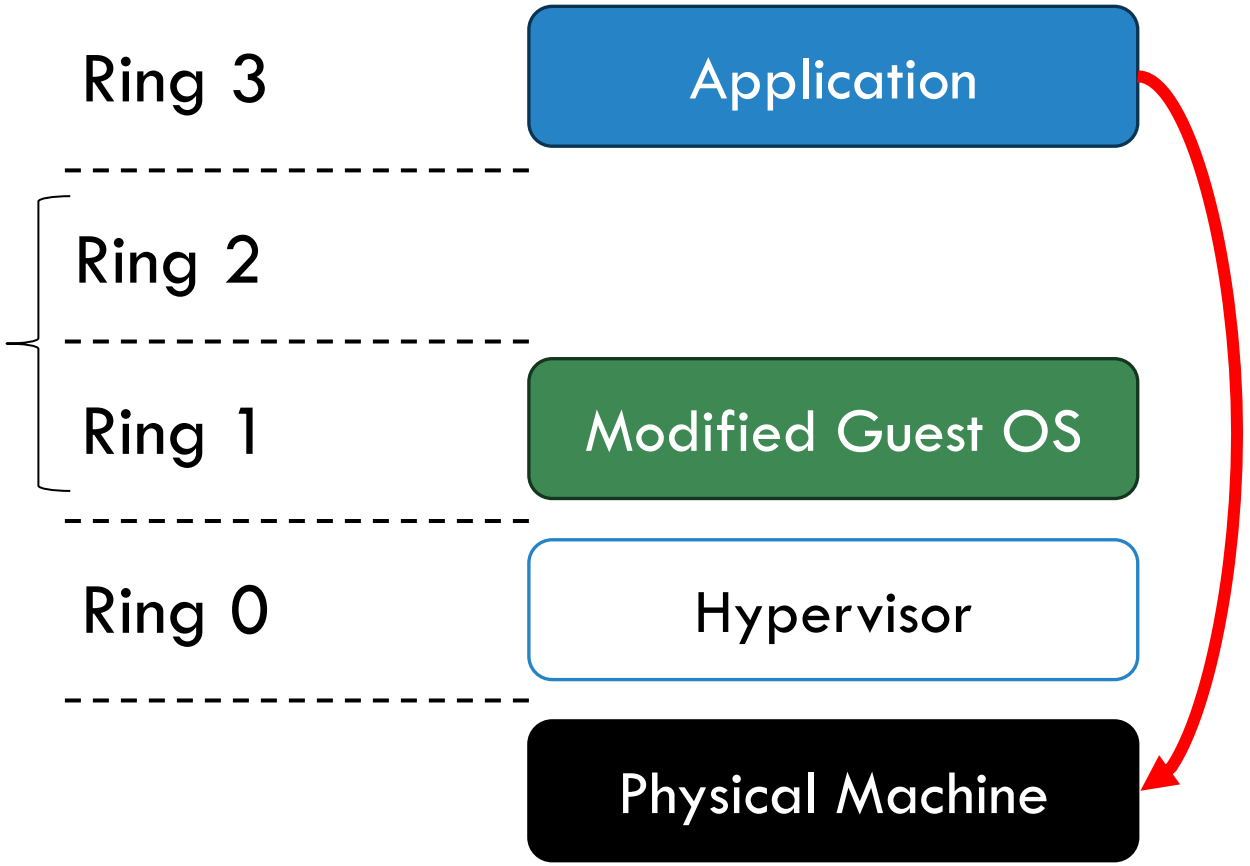


CPU Virtualization

2. Paravirtualization

Modify **guest OS** to be <u>aware</u> it is <u>virtualized</u>

- User-mode instructions are directly executed on CPU



CPU Virtualization

2. **Paravirtualization**

Modify **guest OS** to be <u>aware</u> it is <u>virtualized</u>

- Example: Xen, Denali

Pros ➔ High execution speed similar to non-virtualized systems

Cons ➔ Modified guest OS has poor portability

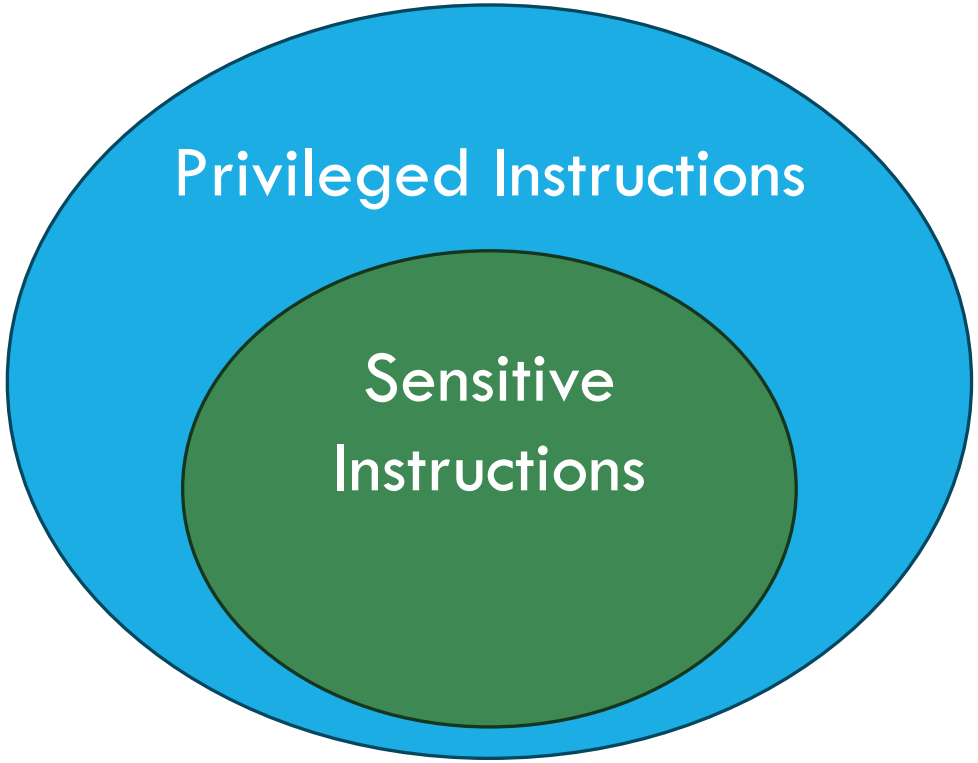
Guest OS must be open-source to be modified

CPU Virtualization

3. **Hardware-assisted virtualization**

Ensure all sensitive instructions trigger trap to the hypervisor

- Remove the need for binary translation

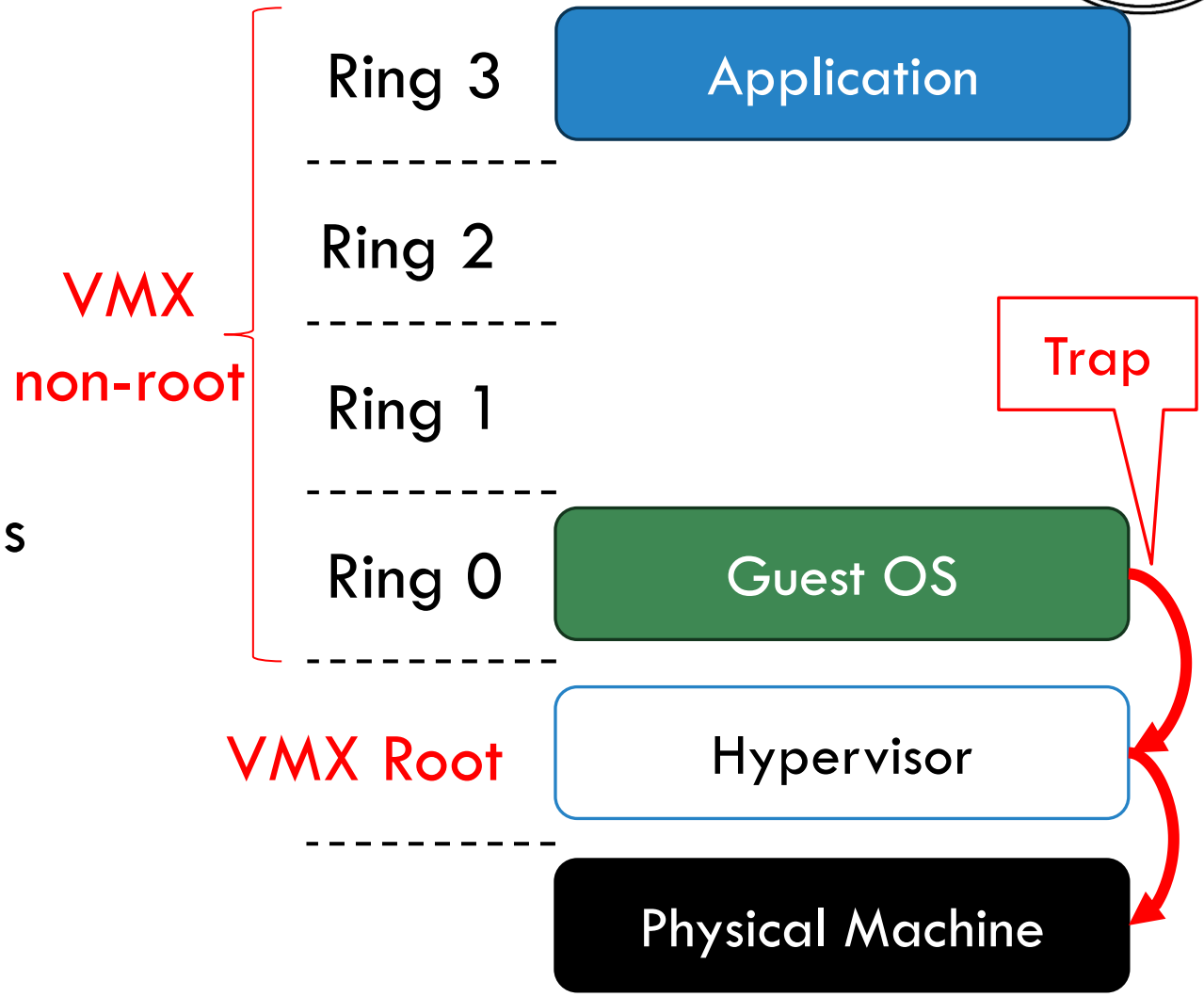


CPU Virtualization

3. **Hardware-assisted virtualization**

Add a new CPU mode feature

- Intel VT-x support 2 modes:
  - VMX root ➔ VMM Operations
  - VMX non-root ➔ Support VM



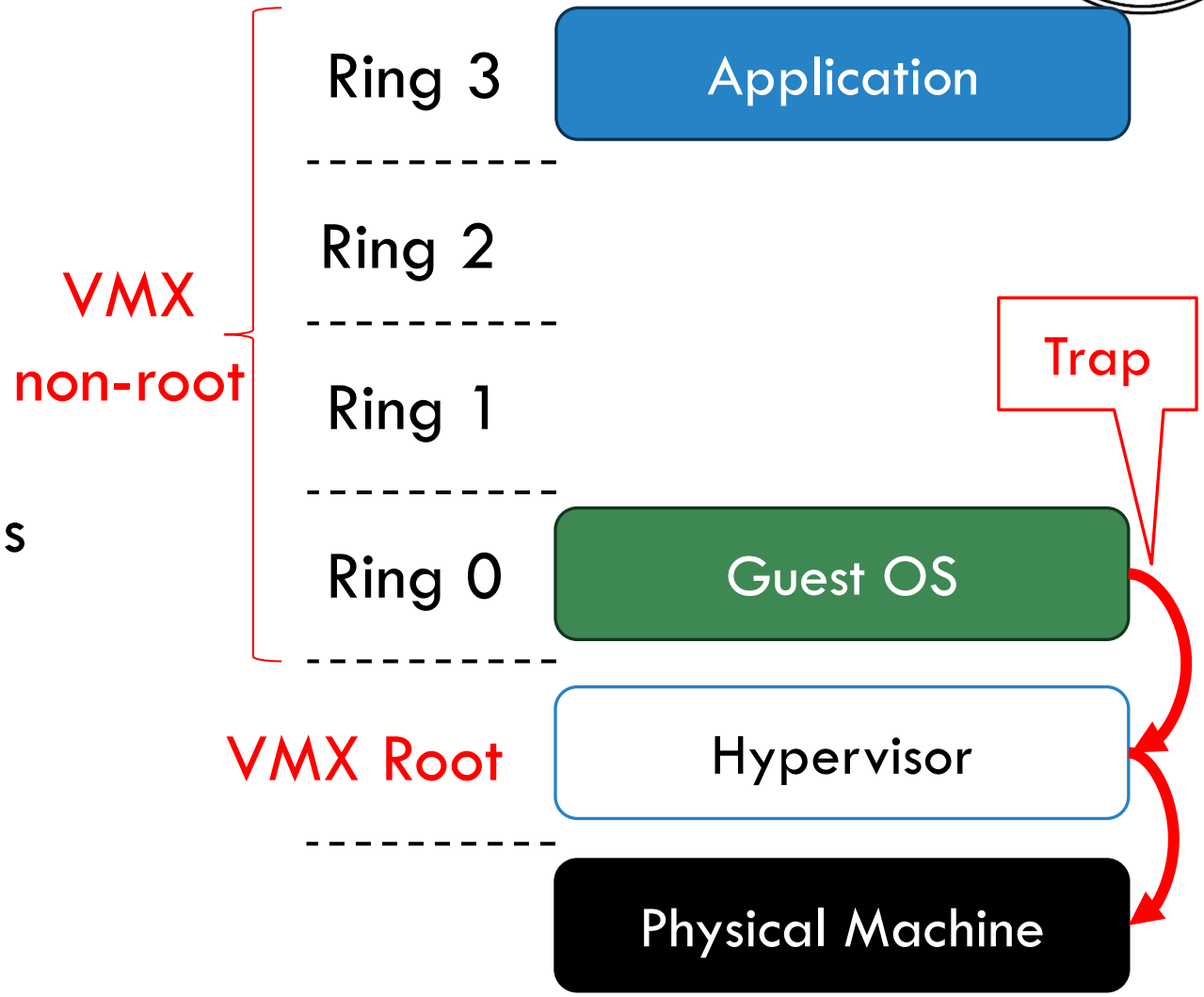
CPU Virtualization

3. **Hardware-assisted virtualization**

Add a new CPU mode feature

- Intel VT-x support 2 modes:
  - VMX root ➔ VMM Operations
  - VMX non-root ➔ Support VM

Pros ➔ Even faster execution



VM Context Switching

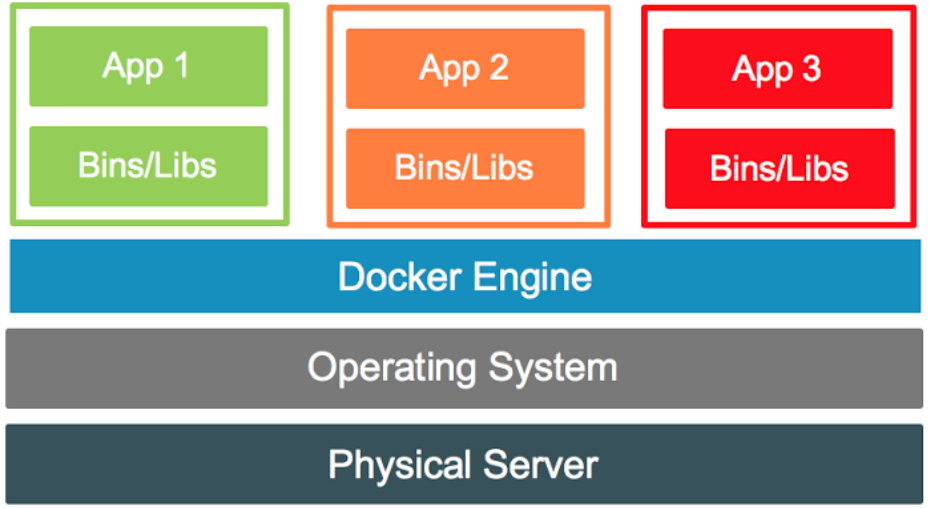
If VM is blocked due to waiting for IO operation or CPU Timeout, VMM will select another VM to operate:

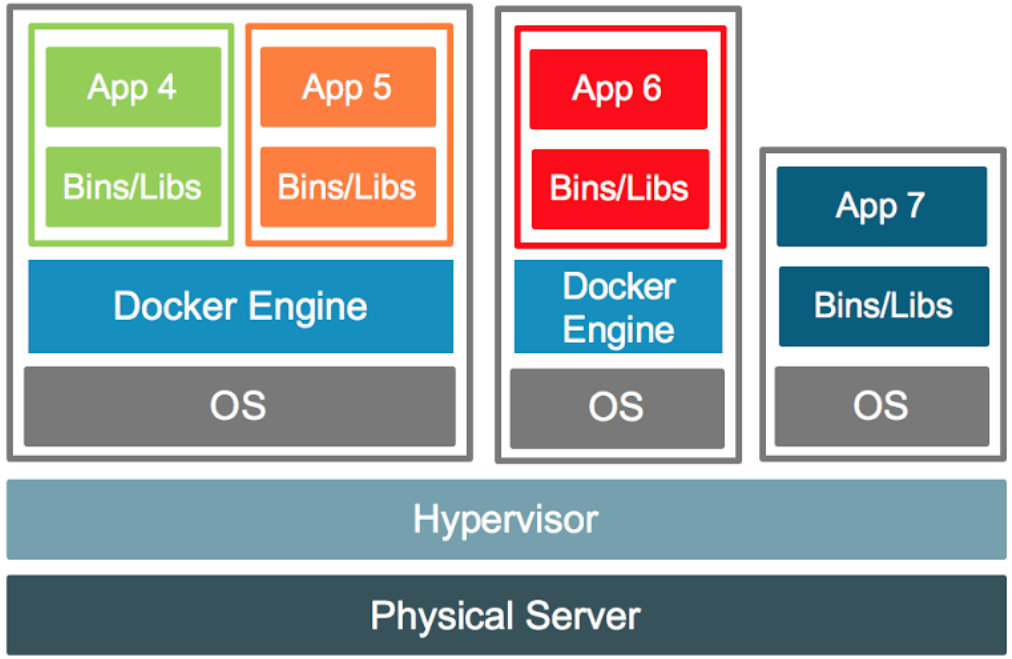
1. Timer Interrupt in running VM.
2. Context switch to Hypervisor.
3. Hypervisor saves state of running VM.
4. Hypervisor determines next VM to execute.
5. Hypervisor sets timer interrupt.
6. Hypervisor restores state of next VM.
7. Hypervisor sets the program counter to timer interrupt handler of next VM.
8. Next VM active

Containers

Lightweight process virtualization

- Linux Process with separate namespace / network space
- Share same kernel of host OS

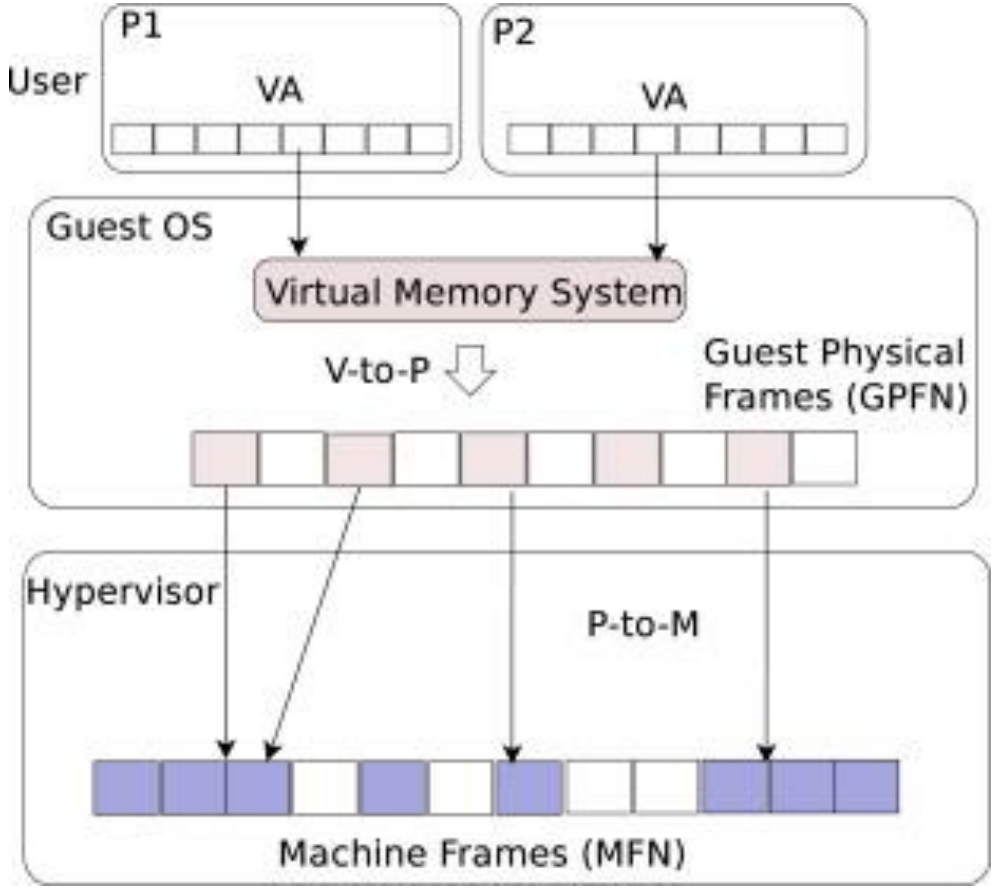




Memory Virtualization

Memory Virtualization

Similar to CPU virtualization, each VM has its own address space

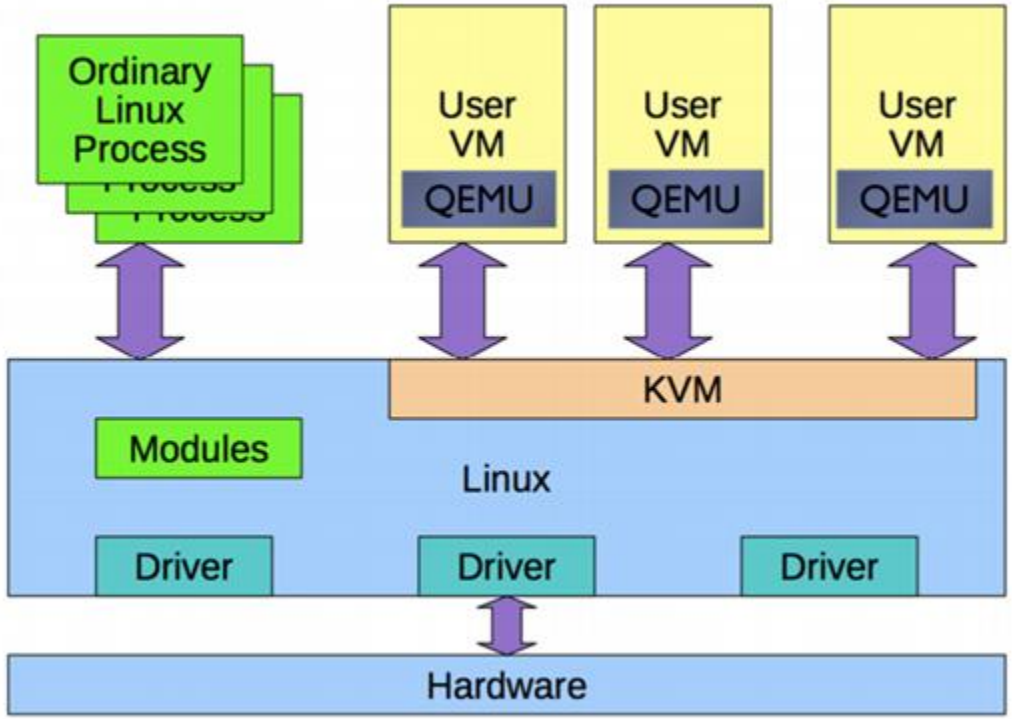


Full Virtualization: KVM

Kernel-based Virtual Machine (KVM)

Open-source VMM

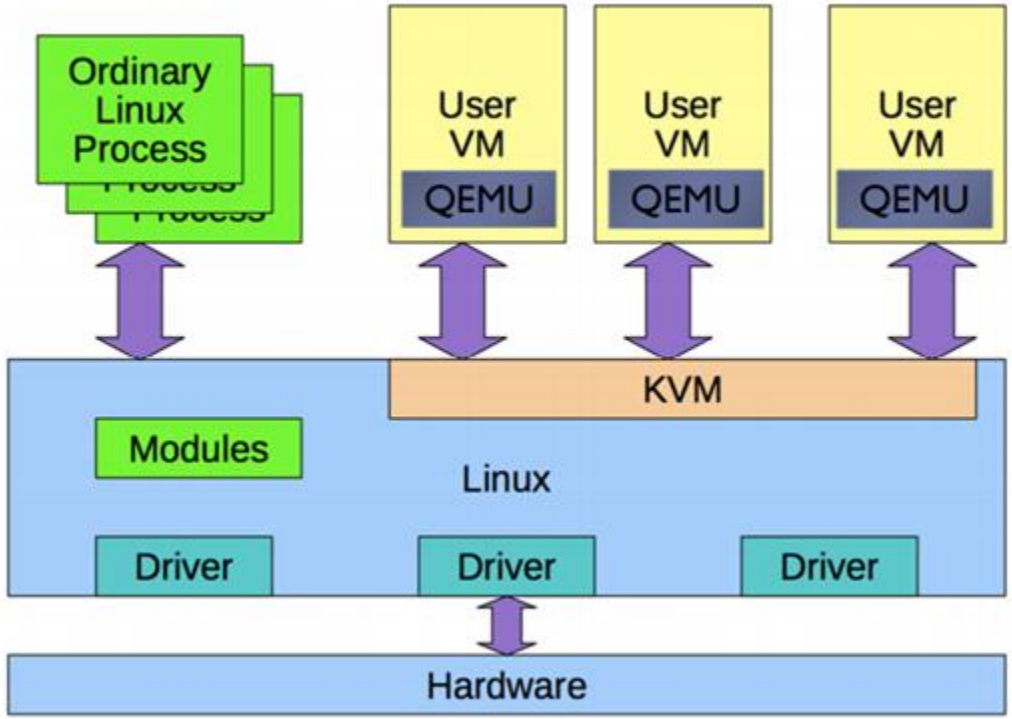
- Transform Linux OS into Bare-Metal VMM
- Provides virtualization infrastructure
- Relies on hardware-assisted virtualization



Kernel-based Virtual Machine (KVM)

Structured as Linux Device /dev/kvm

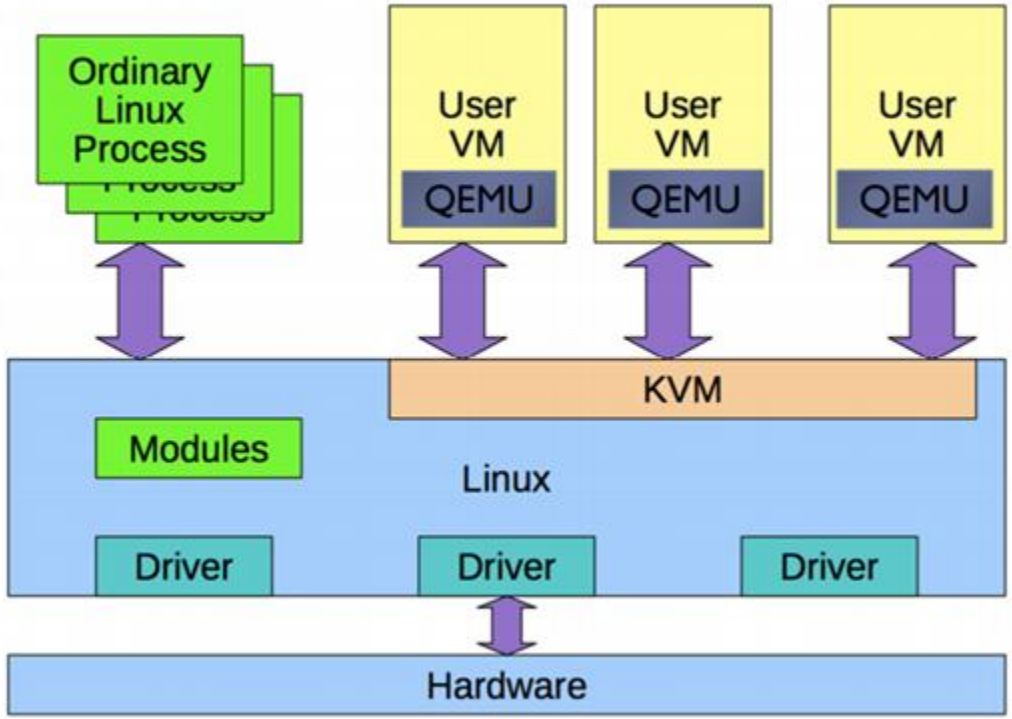
- Has a separate userspace
- Create/run VMs using a set of “ioctl()” commands



Kernel-based Virtual Machine (KVM)

HW dependent operations are specified by separate modules

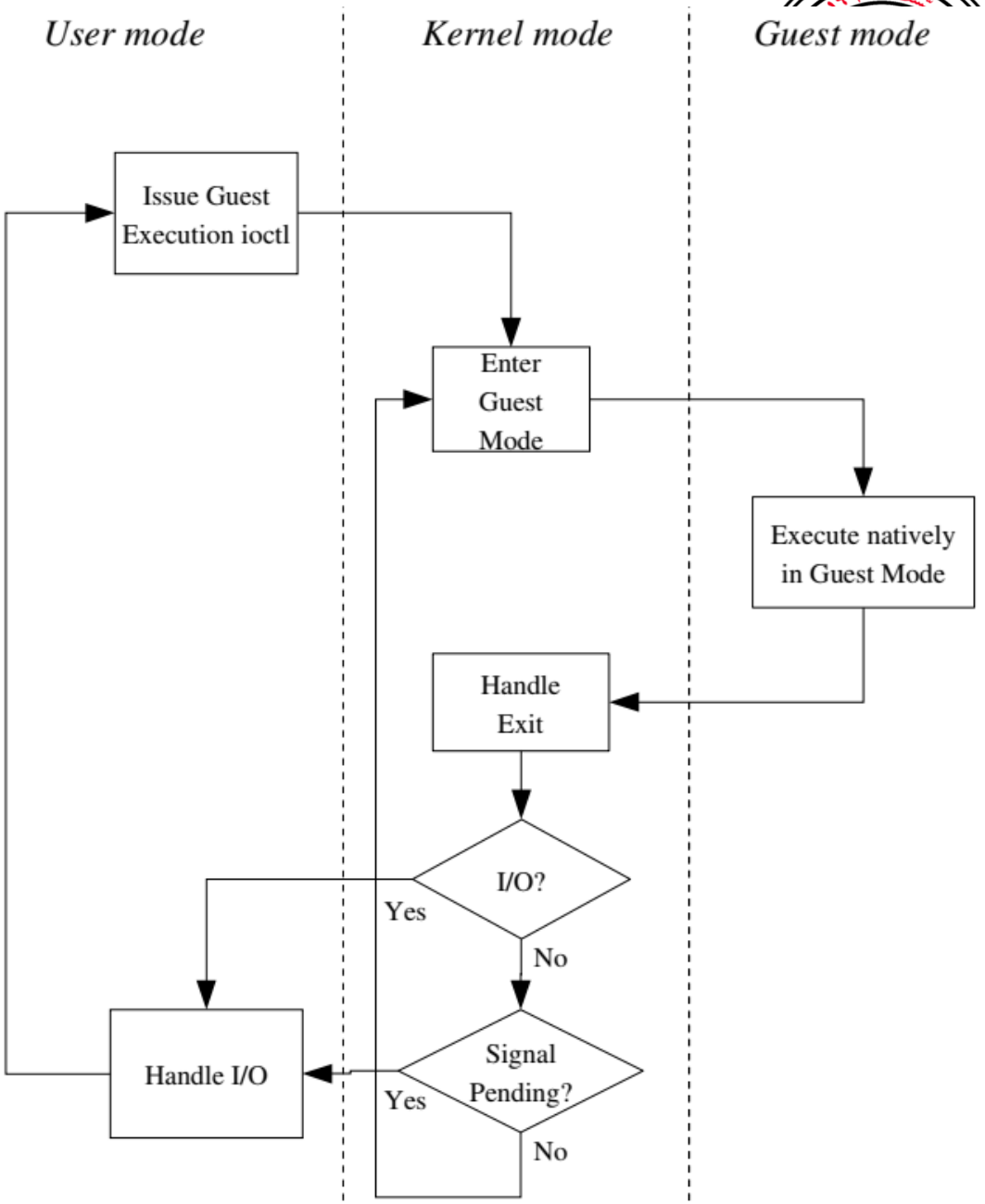
- kvm.ko ➔ x86 ISA
- kvm-intel.ko ➔ Virtualization features added by intel
- kvm-amd.ko ➔ Virtualization features added by AMD



Kernel-based Virtual Machine

Change OS modes

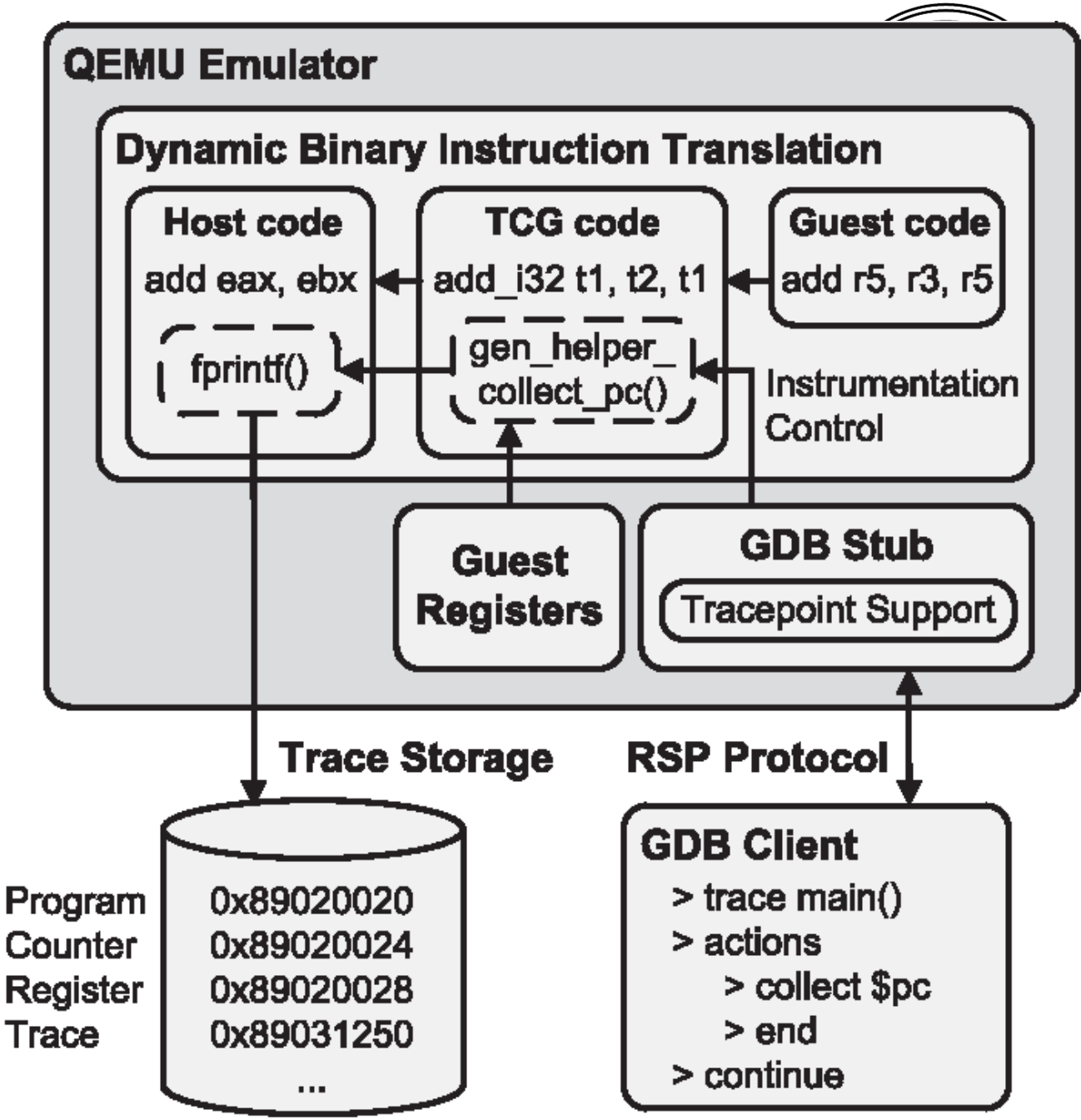
1. User mode
2. Kernel mode
3. Guest mode “VMX non-root”



QEMU Dynamic Translator

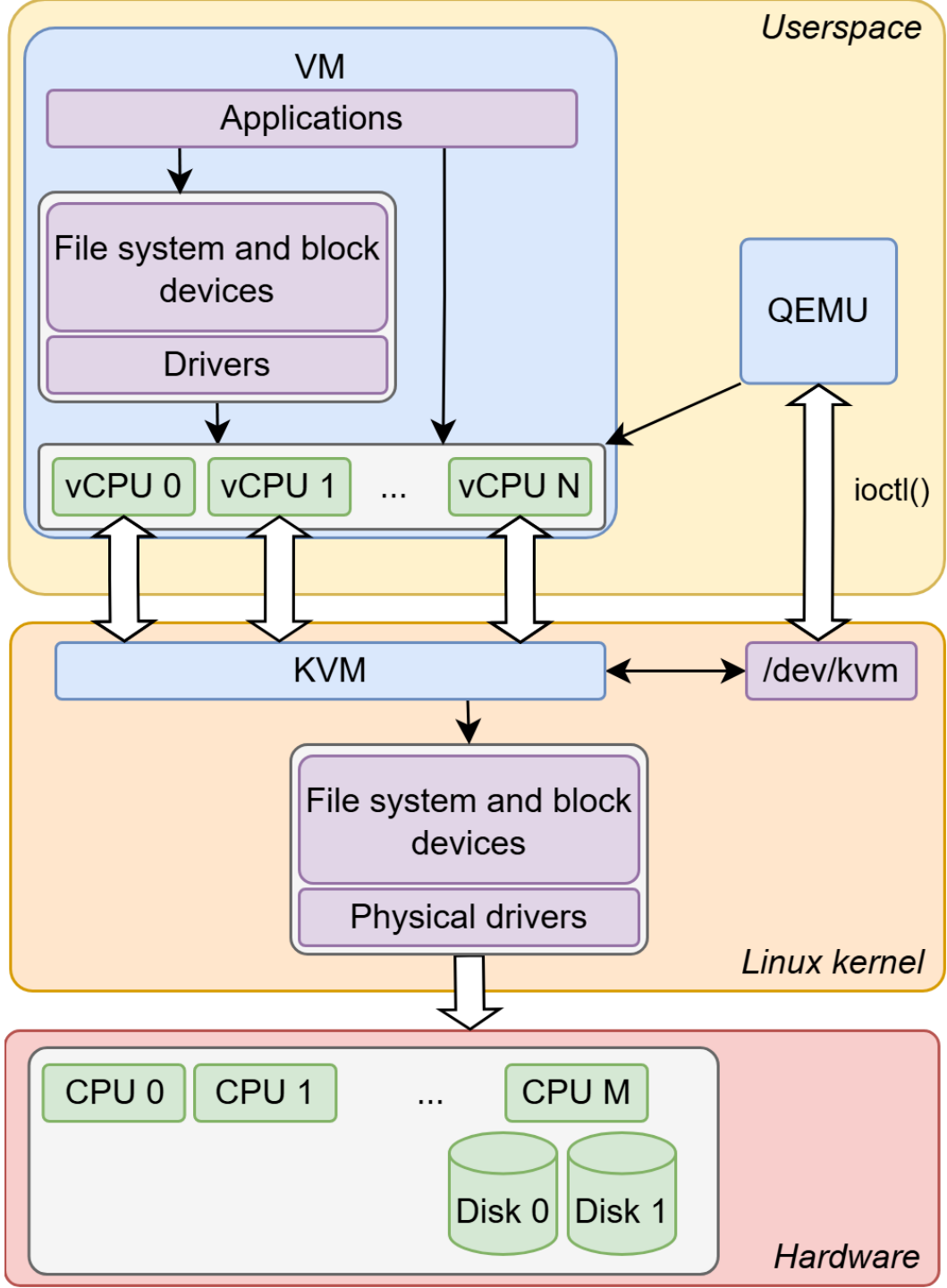
Portable Binary translator

- Emulates multiple guest machine architectures to multiple host machine architectures
- Support Full Virtualization



KVM/QEMU Model

1. KVM operates from within Linux
2. KVM represents vCPUs
3. KVM map vCPUs to physical CPUs and utilize hardware VT
4. vCPUs run in guest mode
5. QEMU initiate VM processes and manage memory space & emulate I/O devices
6. QEMU uses ioctl() to communicate with KVM through /dev/kvm file

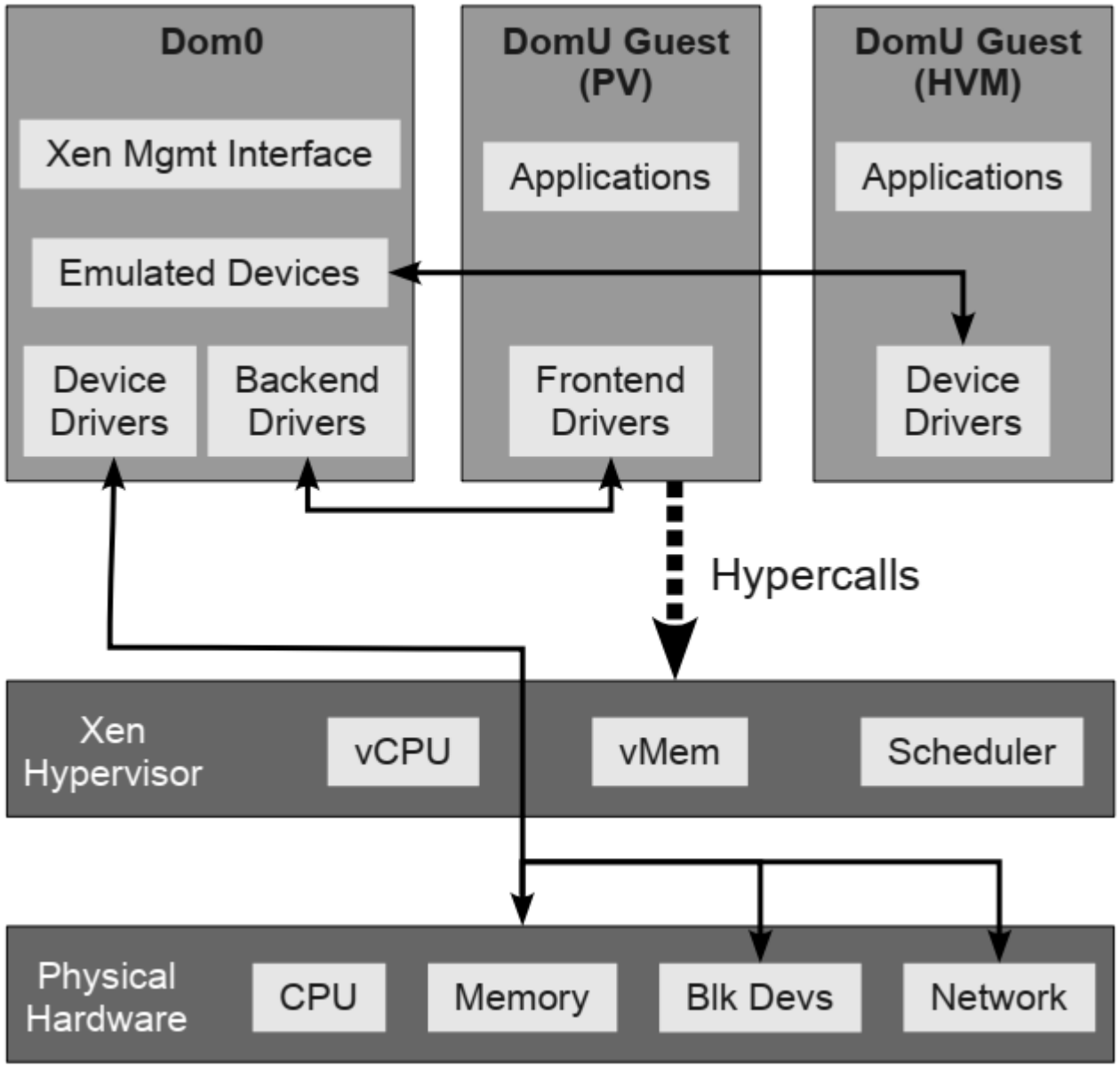


Paravirtualization: Xen

Xen

Open-source VMM

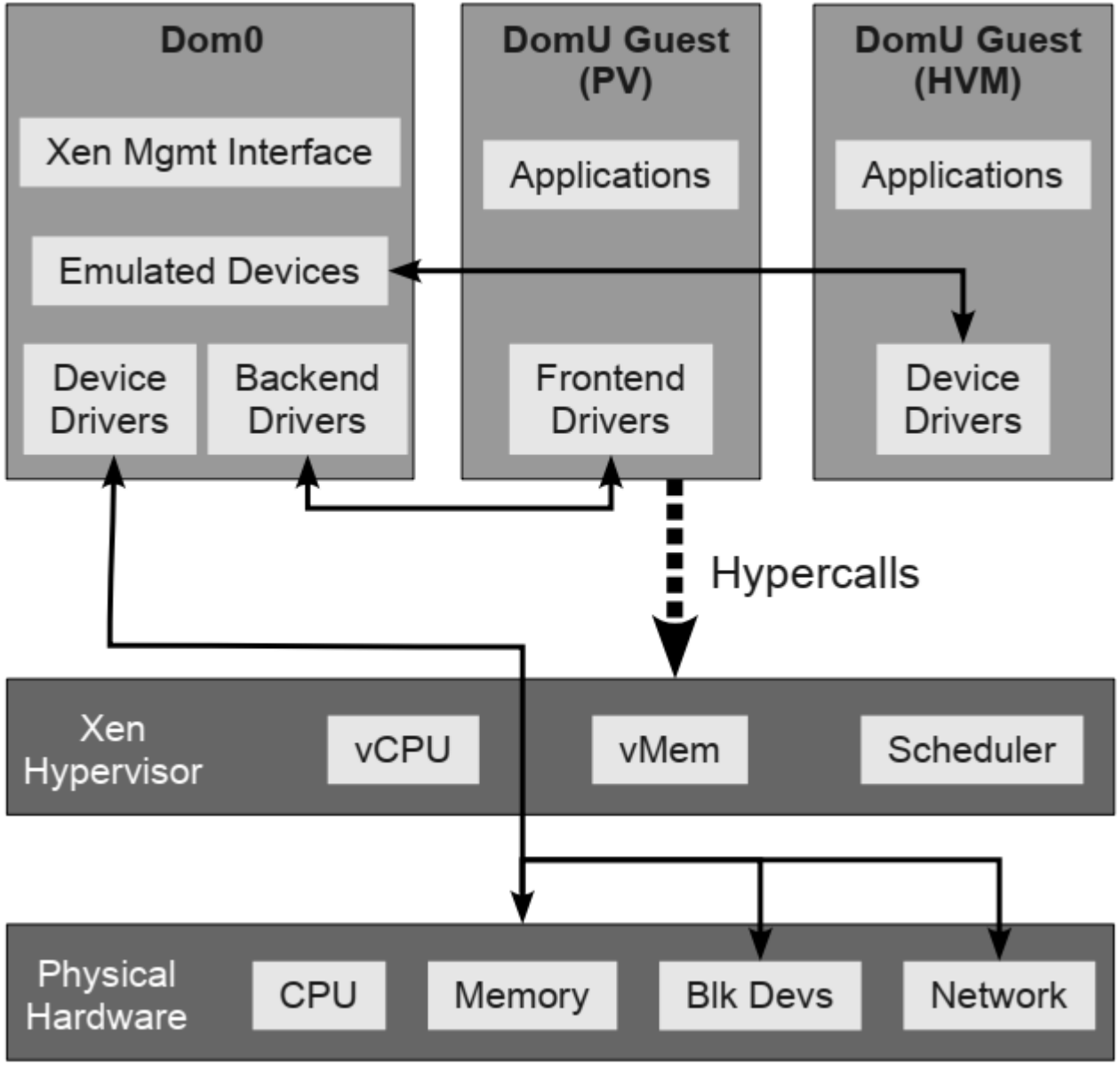
- Bare metal VMM as KVM
- Based on Paravirtualization
- Supports Full Virtualization



Xen

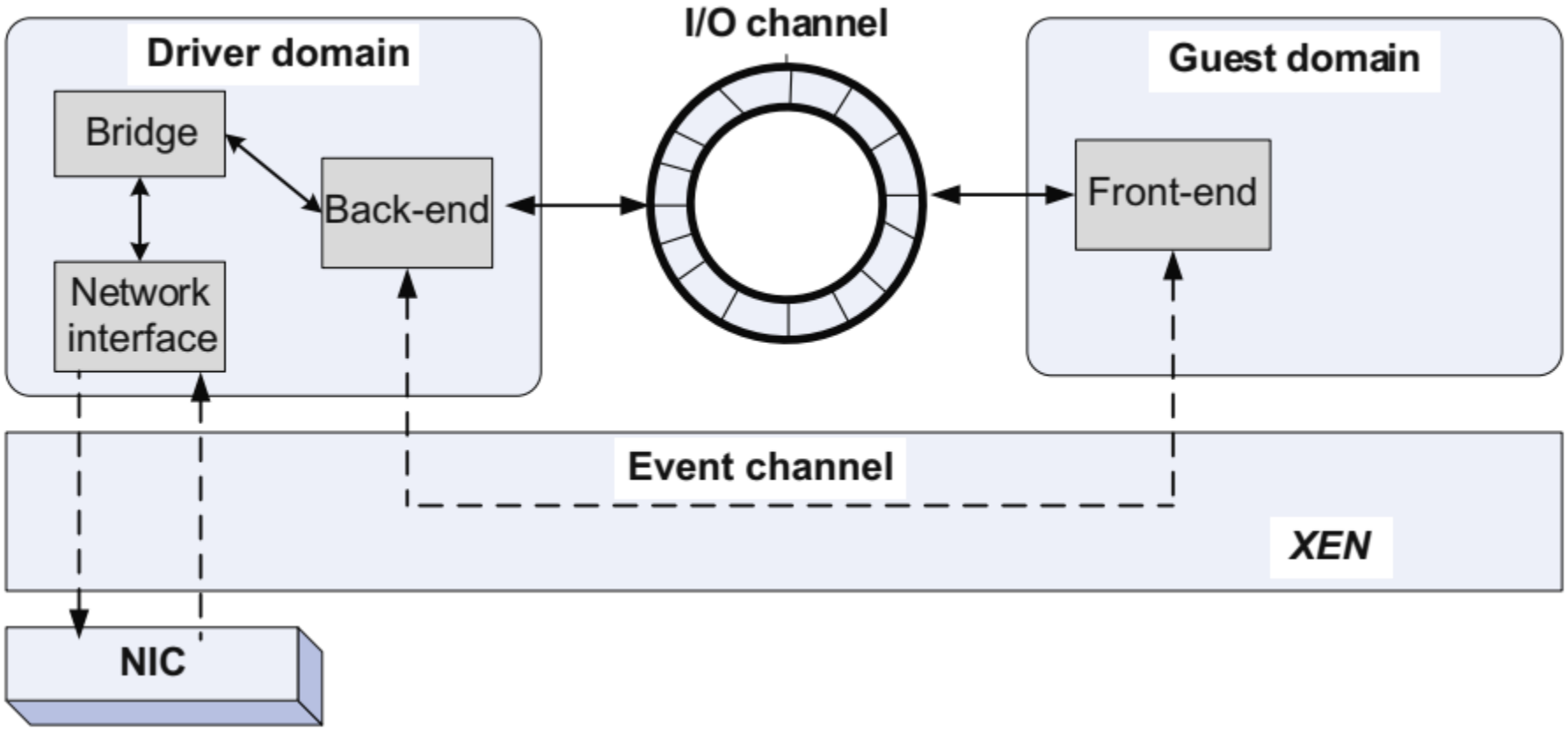
Each VM is referred to as "domains"

- First domain "Dom0" is special
  - Controls Xen VMM
  - Manage other domains
  - Emulate devices



IO Ring

Reduce overhead of VM context switching ➔ Batch IO requests

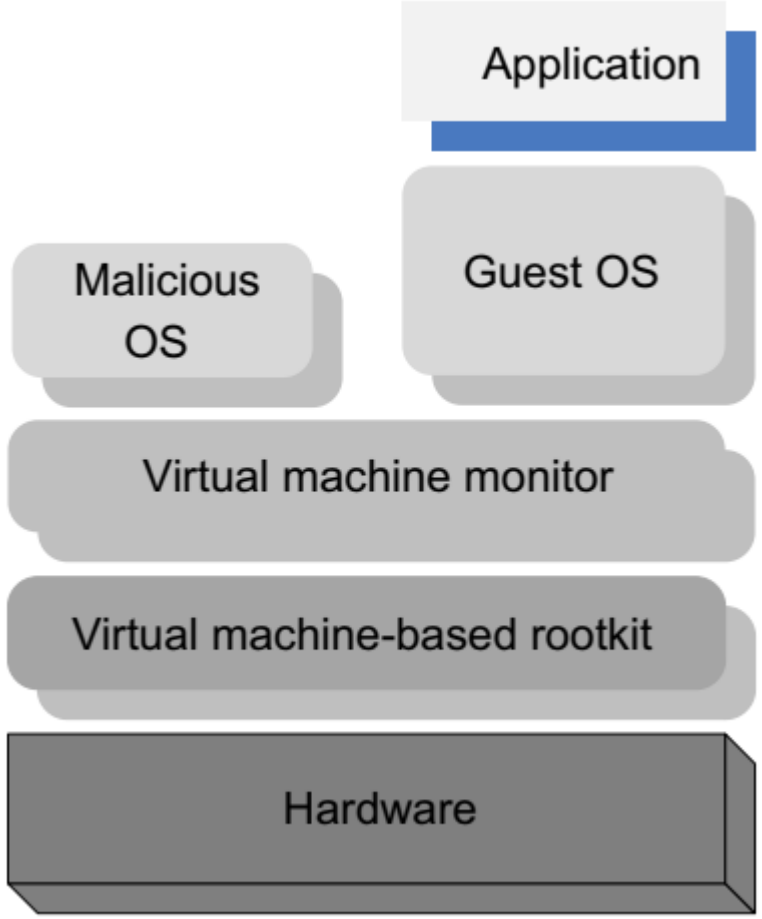


Virtualization Downsides

Rouge Hypervisor

In layered privilege structure, defensive mechanism at a layer can be **disabled** *by malware* in the lower layer

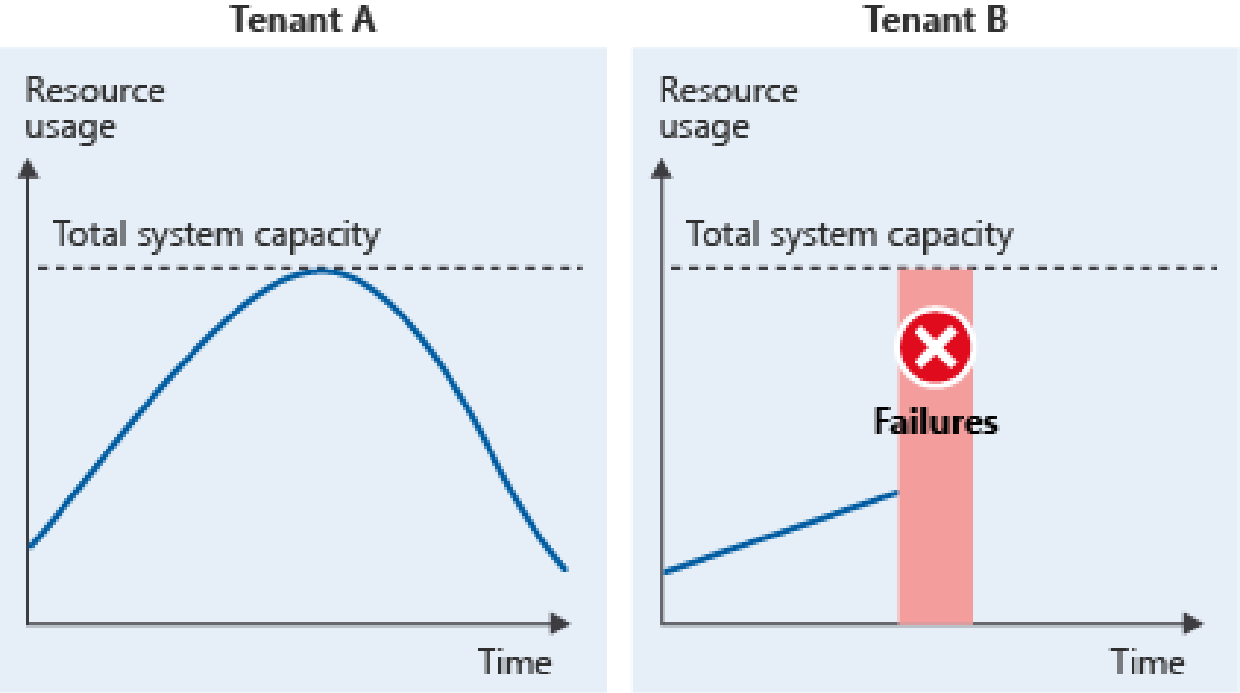
- It is feasible to insert a Virtual-Machine Based Rootkit (VMBR) between physical hardware and an OS or real VMM
  - Create malicious OSs
  - Spy on real guest OS
  - Interfere with guest OS / DoS



Noisy Neighbor

VM running on same host uses all system's resources

- Other VMs can not work properly



Rouge VM

Hidden VM that runs on your machine

- If hacker gets SSH access to server
- A set of simple script could create a VM on VMware ESXi

Resources

- Reference Book: Chapter 5
- 2003, Xen and the Art of Virtualization
  - https://www.youtube.com/watch?v=O938bXFhknU
- 2005, QEMU, a Fast and Portable Dynamic Translator
- 2007, kvm: the Linux Virtual Machine Monitor
- 2020, Analysis of the Intel Pentium's Ability to Support a Secure Virtual Machine Monitor

Thank you